

**A
PROJECT REPORT
ON**

**ADVANCED MACHINE LEARNING APPROACH FOR PHISHING
WEBSITE IDENTIFICATION USING META-ENSEMBLE METHODS**

**In partial fulfilment of the requirements for the award of the
degree of Bachelor of Technology
In**

CSE (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE) SUBMITTED BY

CHANDUJA DEVI.D	(22JD1A4524)
SAI KIRAN.P	(22JD1A4540)
CHUSHMA DEVI SRI.K	(22JD1A4525)
NAGENDRA.K	(22JD1A4521)

Under the guidance of

B.ROJA SRI M.Tech(CSE)

Assistant Professor



DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & DATA SCIENCE)

ELURU COLLEGE OF ENGINEERING AND TECHNOLOGY

DUGGIRALA (V), PEDAVEGI (M), ELURU-534004

APPROVED BY AICTE-NEW DELHI & AFFILIATED TO JNTU-KAKINADA

2022 - 2026

ELURU COLLEGE OF ENGINEERING & TECHNOLOGY

(Affiliated to JNTU-KAKINADA, Approved by AICTE-NEW DELHI)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)



CERTIFICATE

This is to certify that the Project Report entitled **“ADVANCED MACHINE LEARNING APPROACH FOR PHISHING WEBSITE IDENTIFICATION USING META-ENSEMBLE-METHODS”** being submitted in partial fulfilment for the award of the degree of **Bachelor of technology** in **CSE (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)** to the **Jawaharlal Nehru Technological University, Kakinada** is a record of bonafide work carried out by **CHANDUJA DEVI.D (22JD1A4524), SAI KIRAN.P (22JD1A4540), CHUSHMA DEVI SRI.K (22JD1A4525), NAGENDRA.K (22JD1A4521)** under the guidance and supervision.

PROJECT GUIDE

B.ROJA SRI M.Tech (CSE),

Assistant Professor

HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The present project work is the several days study of the various aspects of the project development. During this the effort in the present study, we have received a great amount of help from our **Chairman Sri V.RAGHAVENDRA RAO** and **Secretary V.RAMA KRISHNA RAO** of **Padmavathi group of Institutions**, which we wish to acknowledge and thank from depth of our hearts.

We are thankful to our **Principal Dr. P. BALAKRISHNA PRASAD** for permitting and encouraging us in doing this project.

We are deeply intended to **Sri Dr. K. VADDI KASULU , Associate Professor & Head of the Department**, whose motivation and constant encouragement has led to pursue a project in the field of software development.

We are very much obliged and thankful to our project guide **Sri B.ROJA SRI Assistant Professor**, for providing this opportunity and constant encouragement given by her during the course. We are grateful to her valuable guidance and suggestions during our project work.

Our parents have put our self ahead of themselves. Because of their hard work and dedication, we had opportunity beyond our wildest dreams. Our heartfelt thanks to them for giving us all we ever needed to be successful student and individual.

Finally, we express our thanks to all our other faculty members, classmates, friends and neighbours who helped us for the completion of our project and without infinite love and patience this would never have been possible.

CHANDUJA DEVI.D	(22JD1A4524),
SAIKIRAN.P	(22JD1A4540),
CHUSHMA DEVI SRI.K	(22JD1A4525),
NAGENDRA.K	(22JD1A4521)

ABSTRACT

Phishing attacks are among the most dangerous and rapidly growing cybersecurity threats on the internet. Attackers create fake websites, deceptive URLs, and fraudulent emails to trick users into revealing sensitive information such as login credentials, banking details, and personal data. Traditional security mechanisms, including blacklists, rule-based filters, and heuristic detection systems, are unable to keep pace with modern phishing strategies because attackers continuously modify their techniques and generate new malicious websites. As a result, conventional detection systems suffer from load adaptability and high false-positive rates. To overcome these challenges, this work proposes an intelligent phishing website detection system based on a hybrid meta-learning ensemble approach. The system analysis website features such as URL structure, domain properties, hyperlink behaviour, and webpage characteristics. After preprocessing and feature engineering, multiple machine learning and deep learning classifiers are trained. An Artificial Neural Network (ANN) is used to capture complex global patterns, while a Bagging K-Nearest Neighbour (KNN) model identifies local relationships within the data. Their predictions are combined using Logistic Regression as a meta-classifier to produce the final decision.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
	LIST OF FIGURES	7
	LIST OF ABBREVIATIONS	8
1.	INTRODUCTION	9
	1.1 PURPOSE OF THE PROJECT	10
	1.2 SCOPE OF THE PROJECT	11
	1.3 MODEL DIAGRAMS	12
	1.4 OVERVIEW	12
2.	LITERATURE SURVEY	14
3.	SYSTEM ANALYSIS	16
	3.1 EXISTING SYSTEM & DISADVANTAGES	16
	3.2 PROBLEM STATEMENT	17
	3.3 PROPOSED SYSTEM & ADVANTAGES	17
4.	SYSTEM REQUIREMENTS SPECIFICATION	19
	4.1 FUNCTIONAL REQUIREMENTS	19
	4.2 NON-FUNCTIONAL REQUIREMENTS	22
	4.3 HARDWARE REQUIREMENTS	24
	4.4 SOFTWARE REQUIREMENTS	24
5.	SYSTEM DESIGN	25
	5.1 SYSTEM ENVIRONMENT	25
	5.2 SYSTEM COMPONENTS(MODULES)	29
	5.3 UML/DFD'S	35
	5.4 STATE DIAGRAM	41
	5.5 DEPLOYMENT DIAGRAM	42

6.	SYSTEM STUDY	43
	6.1 ECONOMIC FEASIBILITY	43
	6.2 TECHNICAL FEASIBILITY	43
	6.3 SOCIAL FEASIBILITY	43
7.	IMPLEMENTATION	44
	7.1 MODULES IN PROJECT	44
	7.2 PROCEDURE	45
8.	SOURCE CODE	47
9.	SYSTEM TESTING	60
	9.1 TYPES OF TESTING	60
	9.2 TEST CASES	69
10.	RESULT & OUTPUT SCREENSHOTS	72
11.	CONCLUSION & FUTURE ENHANCEMENT	78
12.	REFERENCES	80
13.	BIBLIOGRAPHY	81

LIST OF FIGURES

Name of the Figure	Page No.
System Architecture	18
Data Flow Diagram	35
Use Case Diagram	37
Class Diagram	38
Sequence Diagram	39
Activity Diagram	40
State Diagram	41
Deployment Diagram	42
Different Types of Software Testing Diagram	60

LIST OF ABBREVIATIONS

ANN – Artificial Neural Network

KNN – K-Nearest Neighbour

SMOTE – Synthetic Minority Oversampling Technique

URL – Uniform Resource Locator

DNS – Domain Name System

SSL – Secure Sockets Layer

API – Application Programming Interface

1. INTRODUCTION

The rapid growth of internet usage and digital transformation has significantly increased online transactions, communication, and data exchange. While these advancements have improved global connectivity and convenience, they have also created opportunities for cybercriminals to exploit vulnerabilities. Among various cyber threats, **phishing attacks** have emerged as one of the most widespread and dangerous forms of cybercrime. Phishing involves the creation of fraudulent websites, deceptive emails, and malicious links designed to trick users into revealing sensitive information such as login credentials, banking details, credit card numbers, and personal data. Phishing websites often imitate legitimate organizations such as banks, e-commerce platforms, and social media services. Attackers continuously modify domain names, URL structures, and webpage designs to bypass traditional security mechanisms. As a result, conventional detection approaches such as blacklist-based filtering, rule-based systems, and heuristic analysis have become insufficient. Blacklist systems can only block previously identified malicious URLs, making them ineffective against newly generated phishing websites. Similarly, rule-based techniques require frequent manual updates and fail to adapt to evolving attack patterns.

To address these limitations, researchers and cybersecurity professionals have increasingly turned to **Machine Learning (ML)** and **Artificial Intelligence (AI)** techniques. Machine learning models can automatically learn patterns from large datasets and identify suspicious characteristics in unseen websites. Unlike static rule-based systems, ML-based systems are dynamic, adaptive, and capable of detecting zero-day phishing attacks. However, relying on a single classification model may still lead to performance limitations such as overfitting, poor generalization, or bias toward certain data patterns.

In recent years, ensemble learning methods have gained popularity in cybersecurity applications due to their ability to combine multiple models for improved performance. Ensemble techniques leverage the strengths of individual algorithms while reducing their weaknesses. Stacking (or stacked generalization) is a powerful ensemble method where multiple base classifiers generate predictions that are combined by a meta-classifier to produce the final output. This project proposes an **Advanced Machine Learning Approach for Phishing Website Identification Using Meta-Ensemble Methods**. The system integrates multiple intelligent models—an Artificial Neural Network (ANN) for capturing complex global patterns, a Bagging K-Nearest Neighbour (KNN) model for identifying local data similarities, and Logistic Regression as a meta-classifier to combine predictions. By integrating these models, the system enhances detection accuracy, reduces false-positive rates, and improves generalization

capability.

The proposed framework not only strengthens phishing detection performance but also provides scalability, real-time analysis, and adaptability to evolving cyber threats. Through advanced feature extraction, balanced dataset handling using SMOTE, and cross-validation techniques, the system ensures reliable and efficient detection of both known and unknown phishing websites. This intelligent hybrid approach contributes significantly to strengthening web security and protecting users from online fraud and identity theft.

1.1 Purpose

The broader goal of this research is to design and develop an intelligent, automated, and scalable phishing website detection system using advanced machine learning techniques and meta-ensemble methods. With the rapid increase in cyber threats, especially phishing attacks, traditional detection mechanisms such as blacklists and rule-based filters have become ineffective against newly generated and sophisticated phishing websites. This research aims to overcome these limitations by introducing an adaptive AI-driven framework capable of detecting both known and unknown phishing attacks with high accuracy.

The primary objective of this project is to enhance phishing detection performance through a hybrid stacking ensemble approach. The system integrates multiple machine learning models including Artificial Neural Network (ANN), Bagging K-Nearest Neighbour (KNN), and Logistic Regression as a meta-classifier. ANN is used to capture complex global patterns in phishing data, Bagging KNN is employed to identify local similarities and neighborhood relationships, and Logistic Regression combines predictions from base learners to generate the final classification output. By leveraging the strengths of multiple algorithms, the system ensures improved generalization, robustness, and reduced false-positive rates.

Another important goal of this research is to implement intelligent feature engineering and data balancing techniques. The system extracts critical features such as URL length, domain age, presence of special characters, SSL certificate status, IP address usage, redirection behavior, and hyperlink analysis. To address the class imbalance problem commonly found in phishing datasets, SMOTE (Synthetic Minority Oversampling Technique) is applied to enhance minority class representation and improve model fairness.

The research also aims to provide real-time phishing detection through a user-friendly web interface. Users can submit URLs or email content for analysis and instantly receive feedback regarding whether the website is legitimate or malicious. This improves accessibility and enhances user trust in the system.

Furthermore, the project seeks to support cybersecurity monitoring and reporting by generating detailed analytical reports including accuracy, precision, recall, F1-score, and confusion matrix. These insights assist administrators in understanding model performance, identifying emerging phishing trends, and updating the system accordingly. Overall, the purpose of this research is to build a high-performance, adaptive, and scalable phishing detection framework that combines intelligent feature extraction, ensemble learning, and real-time classification to strengthen web security and protect users from cyber fraud.

1.2 Scope

The scope of this study involves the design, development, implementation, and evaluation of a machine learning-based phishing website detection system using a meta-ensemble framework. The system focuses on analyzing website-related features and classifying them into phishing or legitimate categories using advanced AI techniques.

One major component of the scope is feature-based phishing detection. The system extracts multiple categories of features including:

- URL-based features (length, presence of “@”, hyphens, number of subdomains)
- Domain-based features (domain age, DNS records, SSL certificate status)
- Content-based features (keyword presence, suspicious scripts)
- Hyperlink-based features (external links, redirections, anchor tags)

These features are preprocessed, normalized, and transformed into structured datasets suitable for machine learning training and testing. Another key area within the scope is the implementation of a stacked ensemble learning model. The system integrates ANN, Bagging KNN, and Logistic Regression in a layered architecture. Cross-validation and hyperparameter tuning are applied to optimize model performance and ensure better generalization across unseen data. The system is deployed using a web-based interface built with HTML, CSS, JavaScript (frontend), and Django (backend). The backend processes user inputs, applies preprocessing and feature extraction, runs the trained ensemble model, and displays the classification result in real time. The database is managed using MySQL for storing user data, model results, and system logs.

Additionally, the system includes administrative functionality for managing users, monitoring system performance, updating datasets, and retraining models when necessary. Performance evaluation metrics such as accuracy, precision, recall, F1-score, ROC-AUC curve, and confusion

matrix are generated to assess detection effectiveness.

The scope also acknowledges certain limitations. The system relies on the quality and diversity of the training dataset. Detection accuracy depends on the extracted features and may vary when attackers develop completely novel evasion techniques. The system is designed as a security assistance tool and does not replace enterprise-level cybersecurity infrastructure. Despite these limitations, the framework is scalable and can be extended in the future to include deep URL embedding models, real-time browser extensions, integration with email servers, or cloud-based threat intelligence platforms.

Overall, the scope establishes a comprehensive AI-driven phishing detection system that improves cybersecurity resilience, enhances real-time protection, and provides a scalable solution for combating evolving phishing threats.

1.3 Model Diagrams

The model architecture consists of the following layers:

1. **Input Layer** – User submits URL or web data.
2. **Data Preprocessing Layer** – Cleaning, normalization, SMOTE balancing.
3. **Feature Extraction Layer** – URL, domain, content, and hyperlink features.
4. **Base Learners Layer** –
Artificial Neural Network (ANN)
Bagging K-Nearest Neighbour (KNN)
5. **Meta-Learner Layer** – Logistic Regression combines base model outputs.
6. **Output Layer** – Final classification (Phishing / Legitimate).
7. **Output Layer** – Final classification (Phishing / Legitimate).

1.4 Overview

This study focuses on developing a robust and intelligent phishing website identification system using advanced machine learning and meta-ensemble techniques. With the increasing dependency on digital platforms for financial transactions, communication, and information

sharing, phishing attacks have become one of the most critical cybersecurity threats. Attackers continuously create fake websites and deceptive URLs that mimic legitimate platforms, making manual detection difficult and inefficient. Traditional security mechanisms rely heavily on blacklists and predefined rules, which are limited in detecting zero-day attacks. These static systems struggle to adapt to evolving phishing strategies. To address this issue, the proposed research introduces a hybrid stacking ensemble learning framework that improves detection accuracy and adaptability.

The system employs Artificial Neural Networks to capture complex nonlinear patterns and high-dimensional relationships within phishing datasets. Bagging KNN is used to analyze local neighborhood similarities and reduce variance through bootstrap aggregation. The predictions from these base models are combined using Logistic Regression, which acts as a meta-classifier to produce a final optimized decision. This multi-layered architecture enhances predictive power and reduces misclassification errors.

The dataset used in this study contains structured phishing and legitimate website features extracted from publicly available cybersecurity repositories. After preprocessing and balancing using SMOTE, the models are trained and evaluated using cross-validation techniques. The hybrid ensemble model demonstrates superior performance compared to individual classifiers, achieving improved accuracy, reduced false positives, and better generalization capability. The system is implemented using Python 3.10 and Django framework, with MySQL database integration. It provides real-time phishing detection through a web-based interface, enabling users to analyze suspicious URLs instantly. The reporting module offers detailed insights into system performance and phishing trends.

In conclusion, this research presents a complete end-to-end AI-based phishing detection framework that integrates intelligent feature extraction, ensemble learning, real-time classification, and performance analytics. The proposed solution contributes to strengthening cybersecurity infrastructure, enhancing user awareness, and reducing financial and identity-related cyber risks in the digital ecosystem.

2. LITERATURE SURVEY

1. Machine Learning-Based Phishing Detection Using URL Features

Author(s): Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. (2009)

Abstract:

This study presents one of the early machine learning approaches for detecting malicious websites using lexical and host-based URL features. The authors propose extracting characteristics such as URL length, token frequency, and domain information to classify phishing and legitimate websites. Supervised learning algorithms including Logistic Regression and Naïve Bayes were evaluated. The results demonstrated that machine learning models outperform traditional blacklist methods in detecting newly generated phishing URLs. However, the study highlights limitations related to feature dependency and the need for continuous retraining to handle evolving phishing tactics.

2. Phishing Website Detection Using Neural Networks

Author(s): Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014)

Abstract:

This research investigates the application of Artificial Neural Networks (ANN) for phishing website detection. The study extracts 30+ website features, including URL-based, HTML-based, and domain-based attributes. The ANN model successfully identifies nonlinear relationships between phishing indicators and achieves higher accuracy compared to rule-based systems. The findings indicate that neural networks can detect zero-day phishing attacks more effectively. However, training complexity and computational overhead are identified as challenges, particularly for large-scale real-time systems.

3. Ensemble Learning for Phishing Detection

Author(s): Aburrous, M., Hossain, M. A., Dahal, K., & Thabtah, F. (2010)

Abstract:

This study explores ensemble learning techniques such as Bagging and Boosting to improve phishing detection accuracy. Multiple base classifiers were combined to enhance

predictive performance and reduce variance. The ensemble approach demonstrated better generalization compared to individual classifiers. The research confirms that combining models increases robustness against diverse phishing strategies. However, model interpretability and computational cost remain important considerations for deployment in real-time environments.

4. Stacking-Based Meta-Learning in Cybersecurity

Author(s): Wolpert, D. H. (1992)

Abstract:

This foundational research introduces the concept of stacked generalization (stacking), where multiple base models are combined using a meta-learner to produce improved predictions. In cybersecurity contexts, stacking enables integration of different learning patterns such as local similarity detection and global pattern recognition. The meta-learning approach reduces overfitting and improves generalization performance. Despite its effectiveness, stacking requires careful cross-validation and model tuning to prevent data leakage and ensure optimal performance.

5. SMOTE for Handling Class Imbalance in Security Datasets

Author(s): Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002)

Abstract:

This study introduces the Synthetic Minority Oversampling Technique (SMOTE) to address class imbalance in datasets. In phishing detection, malicious samples are often fewer than legitimate ones, leading to biased model performance. SMOTE generates synthetic samples of the minority class to balance the dataset. The results show significant improvements in recall and F1-score for minority class detection. However, improper application may introduce noise and increase the risk of overfitting.

6. Deep Learning for Phishing Website Detection

Author(s): Huang, C., & Yao, Y. (2018)

Abstract:

This research applies deep learning architectures such as Convolutional Neural Networks (CNN) and Deep Neural Networks (DNN) for phishing detection. By automatically extracting hierarchical features from URLs and webpage content, deep learning models reduce the need for

manual feature engineering. Experimental results indicate improved detection accuracy and better adaptability to evolving phishing patterns. However, deep learning models require large datasets and higher computational resources, which may limit real-time deployment in resource-constrained environments.

7. Hybrid Machine Learning Approaches for Cyber Threat Detection

Author(s): Sahoo, D., Liu, C., & Hoi, S. C. H. (2017)

Abstract:

This study proposes hybrid machine learning frameworks that integrate multiple classifiers for improved cyber threat detection. By combining models such as KNN, Neural Networks, and Logistic Regression, the system captures both local and global data patterns. The hybrid approach demonstrates higher detection rates and reduced false positives compared to single-model systems. The research emphasizes the importance of meta-learning and cross-validation in ensuring reliable performance across diverse attack scenarios. Limitations include increased system complexity and maintenance overhead.

3. SYSTEM ANALYSIS

3.1 Existing System

Currently, phishing detection is mainly performed using blacklists, rule-based filtering, and heuristic techniques. These systems compare website URLs against stored malicious URL databases. URL matches, the website is blocked. However, attackers frequently create new websites, making blacklist methods ineffective.

Traditional approaches detect only known attacks and cannot recognize newly generated phishing websites. They also rely heavily on manually created rules, which require constant updates. Additionally, these systems produce a high number of false alerts and fail to analyse complex patterns in website behaviour.

Disadvantages of Existing System

- ◆ Detects only known phishing URLs
- ◆ Cannot handle newly created attacks
- ◆ Requires manual updates

- ◆ High false-positive rate
- ◆ Static rule-based detection
- ◆ Poor adaptability to evolving threats
- ◆ Limited feature analysis

3.2 Problem Statement

Existing phishing detection systems rely on blacklist and rule-based techniques, which are limited to detecting known threats. They fail to identify newly generated phishing attacks, leading to low accuracy and high false-positive rates.

3.3 Proposed System

The proposed system introduces a hybrid machine learning phishing detection framework that automatically identifies phishing websites using intelligent models. The system collects website features, preprocesses the data, and applies multiple classifiers. Instead of relying on a single model, the system combines several models using a stacking ensemble approach:

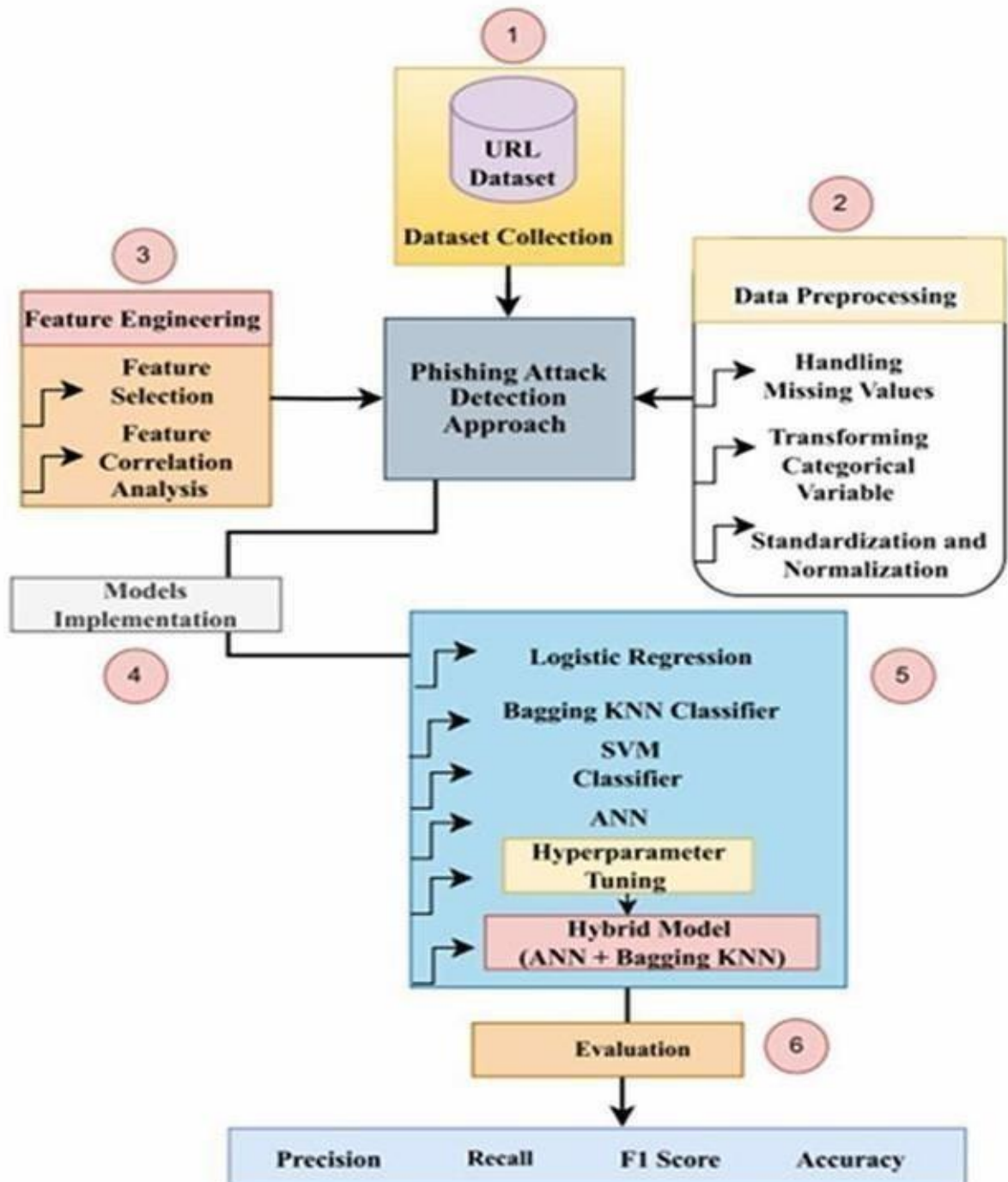
- Artificial Neural Network → detects complex patterns
- Bagging KNN → detects local similarities
- Logistic Regression → combines predictions

This combination improves accuracy and generalization. The model also balances the dataset using SMOTE and evaluates performance using cross-validation. The hybrid model achieves higher detection accuracy and better reliability compared to individual algorithms.

Advantages of Proposed System

- Detects unknown phishing attacks
- High accuracy and precision
- Reduced false alarms
- Automatic learning system
- Real-time website protection
- Scalable and flexible
- Better generalization

System Architecture



4. SYSTEM REQUIREMENTS SPECIFICATIONS

4.1 Functional Requirements

A. User Module

The **User Module** provides an interactive web-based interface where users can analyze suspicious URLs and receive real-time phishing detection results. It ensures ease of access, fast response, and secure interaction with the phishing detection system.

User Registration and Authentication

- The user must be able to register and log in securely using a username and password.
- The system stores user details such as name, email ID, and login credentials securely in the database.
- Authentication ensures that only authorized users can access phishing analysis services.

URL Submission Functionality

- Users can submit suspicious URLs through the web interface.
- The system validates the format of the URL before processing.
- The system checks whether the URL is reachable and properly structured.

Feature Extraction and Preprocessing

- ◆ Once a URL is submitted, the system extracts relevant features such as:
 - ✓ URL length
 - ✓ Presence of special characters (@, -, //)
 - ✓ Number of subdomains
 - ✓ Domain age
 - ✓ SSL certificate status
 - ✓ Redirection behavior
- ◆ The system preprocesses the data (normalization, encoding).
- ◆ SMOTE is applied to balance the dataset during training.

Phishing Classification and Detection

- ◆ The system processes extracted features through the hybrid meta-ensemble model:
- ✓ Artificial Neural Network (ANN)
- ✓ Bagging K-Nearest Neighbour (KNN)
- ✓ Logistic Regression (Meta-classifier)
- ◆ The system classifies the URL as Phishing or Legitimate.
- ◆ Results are displayed along with a confidence score.

Real-Time Detection Output

- ◆ The user receives immediate feedback after URL submission.
- ◆ The result includes:
- ✓ Classification status
- ✓ Probability score
- ✓ Basic explanation (e.g., suspicious domain age or abnormal URL length).

Report Generation and History

- Users can download a phishing analysis report.
- Reports include prediction result, confidence level, and extracted feature summary.
- The system maintains a history of previously analyzed URLs for each user.

Feedback and Interaction

- Users can provide feedback on prediction accuracy.
- Feedback data is stored for model improvement and retraining.

Help and Support

- The system provides a help section with FAQs and usage instructions.
- Contact support option is available for technical issues.

B. Admin Module

The **Admin Module** is designed for system administrators and cybersecurity analysts responsible for maintaining datasets, retraining models, and monitoring system performance.

Admin Authentication

- Admin must log in securely using role-based authentication.
- Only authorized administrators can access sensitive system controls.

Dataset Management

- Admin can upload, update, or remove phishing and legitimate URL datasets.
- Dataset labeling and feature verification are supported.
- Data preprocessing operations can be triggered manually.

Model Training and Evaluation

- ◆ Admin can initiate training or retraining of ANN, Bagging KNN, and Logistic Regression models.
- ◆ Cross-validation and hyperparameter tuning options are available.
- ◆ The system displays performance metrics including:
 - ✓ Accuracy
 - ✓ Precision
 - ✓ Recall
 - ✓ F1-Score
 - ✓ ROC-AUC Curve

Ensemble Model Management

- Admin can modify stacking configurations.
- Meta-classifier parameters can be adjusted.
- The system allows comparison between individual models and ensemble performance.

User Management

- Admin can monitor registered users.
- Access logs and URL submission history can be reviewed.
- User accounts can be activated, deactivated, or removed.

Performance Monitoring and Analytics

- The system provides dashboard analytics showing:

- ✓ Number of URLs analyzed
- ✓ Phishing detection trends
- ✓ Model accuracy over time
- Admin can export analytical reports for research or monitoring purposes.

System Logs and Maintenance

- The system stores logs of user interactions and model predictions.
- Backup and recovery mechanisms are available.
- Error monitoring and debugging tools are integrated.

Security and Data Privacy Control

- Data encryption is implemented for stored user and dataset information.
- Role-based access control prevents unauthorized access.
- Secure communication using HTTPS protocol is enforced.
- Regular vulnerability checks and system audits are supported.

4.2 Non-Functional Requirement

1. Performance

Performance is critical for real-time phishing detection. The system must provide classification results within a few seconds after URL submission. The ANN, Bagging KNN, and Logistic Regression models are optimized for fast inference. Backend processes are implemented efficiently using Python and Django to reduce latency. The system must handle multiple concurrent user requests without performance degradation.

2. Scalability

The system is designed with modular architecture, enabling scalability as user demand increases. Cloud deployment readiness allows horizontal scaling of servers and databases. The architecture supports future integration with browser extensions, email filters, or enterprise cybersecurity platforms.

3. Reliability and Robustness

The system must function consistently under various operating conditions. Error-handling mechanisms manage invalid URLs and system failures gracefully. Model predictions are validated with confidence thresholds. Continuous retraining ensures adaptability to emerging phishing techniques.

4. Usability

The web interface is simple, intuitive, and user-friendly. Users can submit URLs easily and understand results clearly. The system provides clean visualization of results and downloadable reports. Compatibility across different browsers and devices is ensured.

5. Maintainability

The architecture follows modular development principles. Machine learning models, feature extraction modules, and database components are independent. This enables easy updates, debugging, and enhancement without affecting the entire system. Proper documentation and logging ensure long-term maintainability.

6. Interoperability

The system supports RESTful APIs and JSON data exchange formats. It can integrate with external cybersecurity tools, SIEM systems, or browser extensions. Future expansion to cloud-based threat intelligence platforms is supported.

7. Security

Security is a major requirement due to the sensitive nature of cybersecurity data. The system implements:

- Secure authentication mechanisms
- Encrypted database storage
- HTTPS communication
- Role-based access control
- Regular vulnerability assessments

User privacy and data integrity are maintained at all times, ensuring trust and system credibility.

4.3 Hardware Requirements

MINIMUM (Required for Execution)		MY SYSTEM (Development)
System	Pentium IV 2.2 GHz	i3 Processor 5 th Gen
Hard Disk	20 Gb	500 Gb
Ram	1 Gb	4 Gb

4.4 Software Requirements / Technologies Used

Operating System	Windows 10/11
Development Software	Python 3.10
Programming Language	Python
Front End Technologies	HTML5, CSS3, Java Script
Back End Technologies or Framework	Django
Database Language	SQL
Database (RDBMS)	MySQL
Web Server or Deployment Server	Django Application Development Server

5. SYSTEM DESIGN

5.1 System environment

What is Python

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

1. Machine Learning
2. GUI Applications (like Kivy, Tkinter, PyQt etc.)
3. Web frameworks like Django (used by YouTube, Instagram, Dropbox)
4. Image processing (like Opencv, Pillow)
5. Web scraping (like Scrapy, BeautifulSoup, Selenium) Test frameworks
6. Multimedia

What is Machine Learning

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of **Models of data**.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Types of Machine Learning

Supervised Learning – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.

Unsupervised Learning – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.

Semi-supervised Learning – This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.

Reinforcement Learning – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. It support

Multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules Used in Project

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google. TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions

- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities
- Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a 37 wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits.

Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of **functions familiar to MATLAB users.**

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

5.2 System Components

1. User Module

This module allows users to access the phishing detection system, upload URLs or email data for analysis, and view detection results. It provides real-time feedback on whether a link or message is legitimate or phishing-based, ensuring user-friendly interaction and protection.

2. Admin Module

The admin manages users, datasets, and the overall operation of the phishing detection system. This module oversees data preprocessing, model updates, system monitoring, and the addition of new phishing patterns or attack signatures.

3. Data Preprocessing Module

Responsible for cleaning, transforming, and normalizing input data (URLs, emails, or web content). It also handles **SMOTE oversampling** to balance class distribution and prepares the dataset for model training and testing.

4. Feature Extraction and Engineering Module

Extracts relevant features from input data such as URL length, domain, age, SSL certificate status, and keyword presence. It enhances the quality of input data for more accurate phishing detection.

5. Hybrid Model Training Module

Implements the **stacked ensemble learning model combining ANN, Bagging KNN, and Logistic Regression**. It handles model training, optimization, cross-validation, and hyperparameter tuning for improved prediction accuracy.

6. Phishing Detection Module

This is the core module that classifies data in real-time using the trained ensemble model. It applies the meta-learning strategy to detect phishing attempts based on global and local data patterns.

7. Reporting and Analysis Module

Generates reports summarizing detection accuracy, precision, recall, and overall system performance. It provides insights into phishing trends, model behavior, and system efficiency for both users and administrators.

This is the core module that classifies data in real-time using the trained ensemble model. It applies the meta-learning strategy to detect phishing attempts based on global and local data patterns.

8. Reporting and Analysis Module

Generates reports summarizing detection accuracy, precision, recall, and overall system performance. It provides insights into phishing trends, model behavior, and system efficiency for both users and administrators.

ALGORITHMS

ARTIFICIAL NEURAL NETWORK (ANN)

- Overview

- ANN is a machine learning algorithm inspired by the human brain and is used to detect complex patterns in phishing data

- Structure

- It consists of input layer, hidden layers, and output layer where the input layer receives features like URL length and domain details and hidden layers process data using weights and activation functions

- Working

- The input data is passed through multiple layers where computations are performed and activation functions help in learning non-linear patterns and the output layer produces the final classification

- Training

- The model is trained using labeled datasets and backpropagation is used to reduce error by adjusting weights and improving accuracy over time

- Advantages

- It provides high accuracy in detecting phishing patterns and is suitable for handling large datasets and complex relationships

K-NEAREST NEIGHBOUR (KNN)

- Overview

- KNN is a supervised learning algorithm that classifies data based on similarity between input and existing data points

- Working Principle

- It calculates the distance between the input URL and all data points in the dataset and selects the nearest neighbours based on the value of K and uses majority voting for classification

- Distance Measures

- It commonly uses Euclidean distance and other distance metrics to measure similarity between data points

- Features

- It does not require a separate training phase and is simple to implement but requires more computation during prediction

- Advantages

- It is effective for identifying similar phishing patterns and works well for smaller datasets and similarity-based classification

LOGISTIC REGRESSION

- Overview

- Logistic Regression is used for binary classification and predicts the probability of a URL being phishing or legitimate

- Working

- It uses a sigmoid function to convert input features into probability values between zero and one and compares the result with a threshold to classify the output

- Training

- The model learns from labeled data and uses optimization techniques to minimize error and improve prediction accuracy

- Features

- It is simple, efficient, and easy to interpret and works well when there is a linear relationship between input and output

- Advantages

- It requires less computational power and provides fast and reliable results with probability-based output

META-ENSEMBLE METHOD

- Overview

- The meta-ensemble method combines multiple machine learning models to improve the overall performance and accuracy of the system

- Working

- It collects predictions from ANN, KNN, and Logistic Regression and combines them using techniques such as majority voting or weighted averaging to produce a final decision

- Techniques
 - Common techniques include majority voting and weighted averaging where better performing models can be given higher importance
- Advantages
 - It reduces errors from individual models and increases reliability and robustness of the system and improves prediction accuracy

WORKING OF COMBINED ALGORITHMS

- Input Processing
 - The user enters a URL and the system extracts important features from the input and prepares the data for analysis
- Model Execution
 - The processed data is given to all machine learning models and each model independently analyzes the input based on its algorithm
- Result Combination
 - The outputs from all models are collected and passed to the meta-ensemble module which combines the predictions
- Final Output
 - The system generates the final classification as phishing or legitimate and displays the result to the user

PERFORMANCE FACTORS

- Data Quality
 - High-quality datasets improve the performance and accuracy of the models and proper preprocessing is required
- Model Efficiency
 - Proper parameter tuning and combining multiple models increases system accuracy and efficiency
- Evaluation
 - Performance metrics such as accuracy, precision, and recall are used to evaluate the effectiveness of the system and identify improvements.

Feature selection plays an important role in improving the performance of the phishing detection system. The system considers features such as URL length, domain age, presence of HTTPS, number of subdomains, and special characters. These features help the algorithms to distinguish between phishing and legitimate websites more effectively and reduce unnecessary data processing.

Data preprocessing is essential to prepare the collected data for model usage. The raw data may contain missing values or unwanted information, which is handled by cleaning and normalizing the dataset. This step ensures that the input given to the algorithms is accurate and consistent, leading to better prediction results. The dataset used in the system is divided into training and testing data. The training data is used to build the machine learning models, while the testing data is used to evaluate their performance. This helps in checking how well the model can handle new and unseen data.

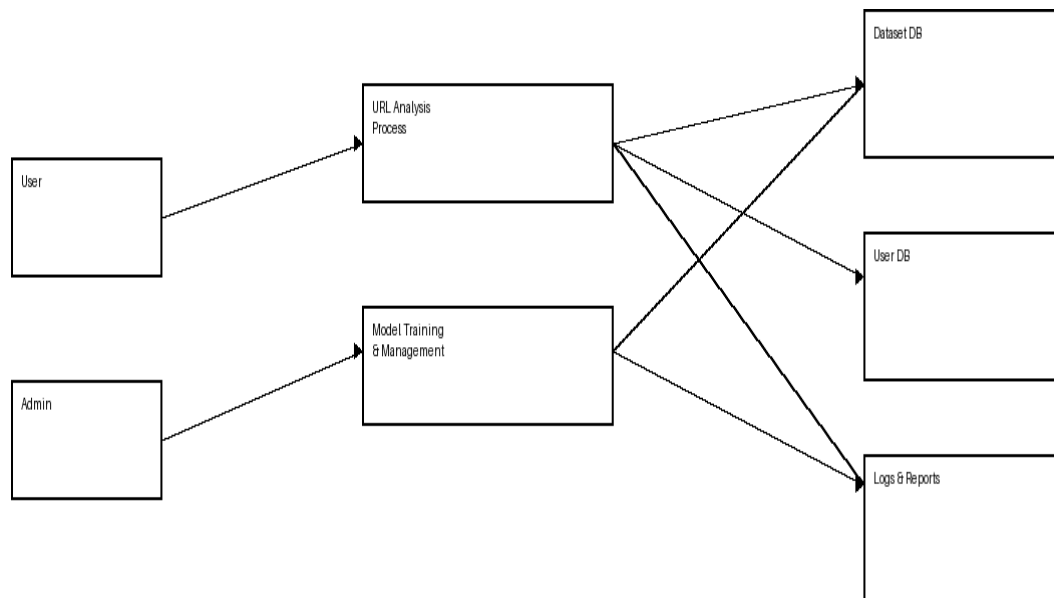
Evaluation metrics are used to measure the performance of the system. Accuracy shows how many predictions are correct, precision indicates the correctness of phishing predictions, and recall shows how many actual phishing websites are detected. These metrics help in analyzing and improving the system.

Overfitting and underfitting are common challenges in machine learning. Overfitting happens when the model performs well on training data but poorly on new data, while underfitting occurs when the model fails to learn patterns properly. These issues are handled using proper parameter tuning and validation techniques. The system is flexible and can be updated with new data and improved models. As phishing techniques change over time, the system can be retrained to maintain its accuracy. This makes the system reliable and suitable for real-world applications.

Security is also an important aspect of the system. User data and results are stored securely, and access is provided only to authorized users. This ensures data safety and builds trust among users.

5.3 Data Flow Diagram

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.



Overview

DFD is a graphical representation of the system that shows how data flows from input to output, It helps in understanding the overall working of the phishing detection system.

Components

Includes processes, data flow, data stores, and external entities.

Shows how data moves between user, system modules, and database.

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

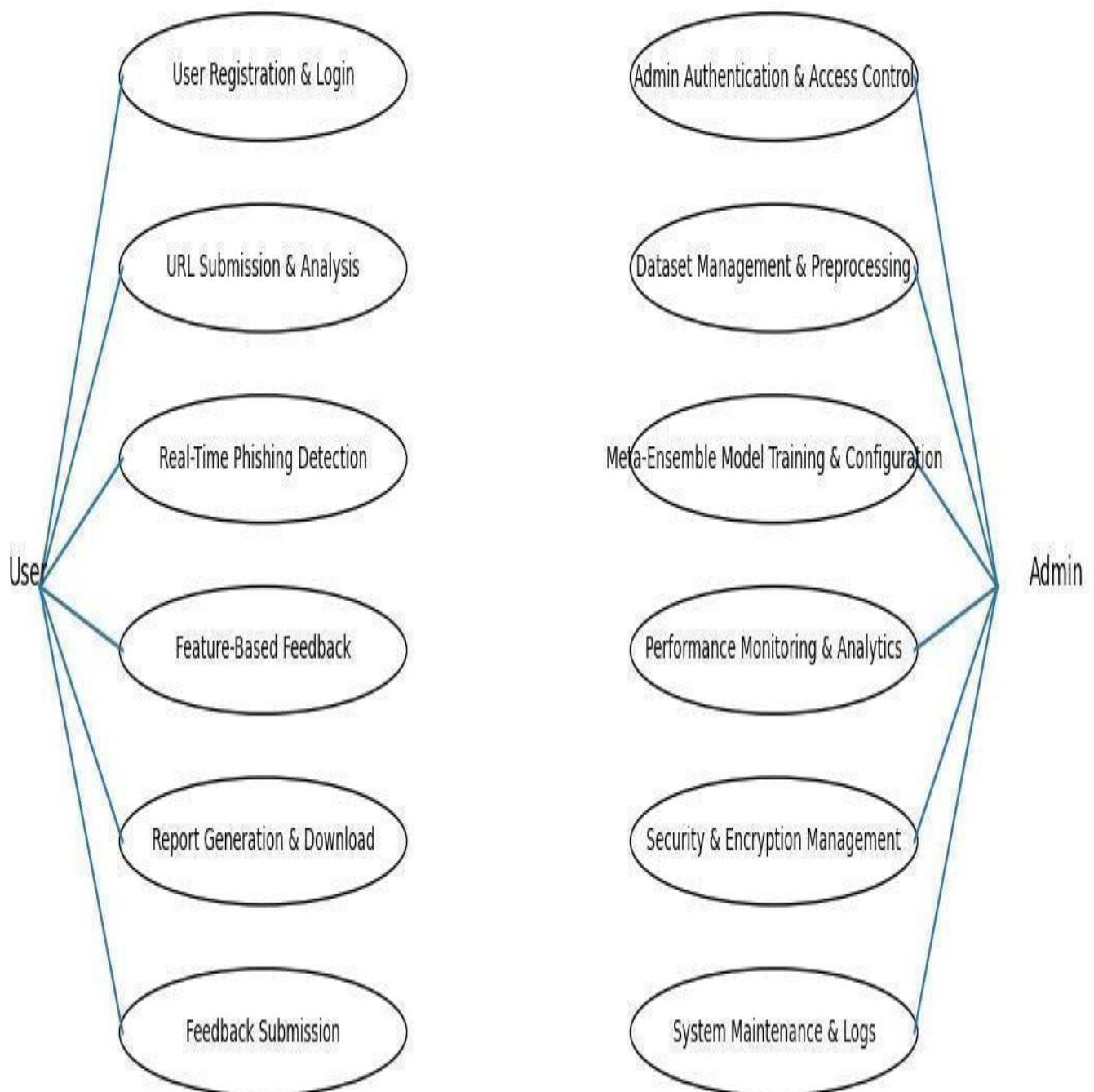
GOALS

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as co laborations, frameworks, patterns and components.
7. Integrated best practices.

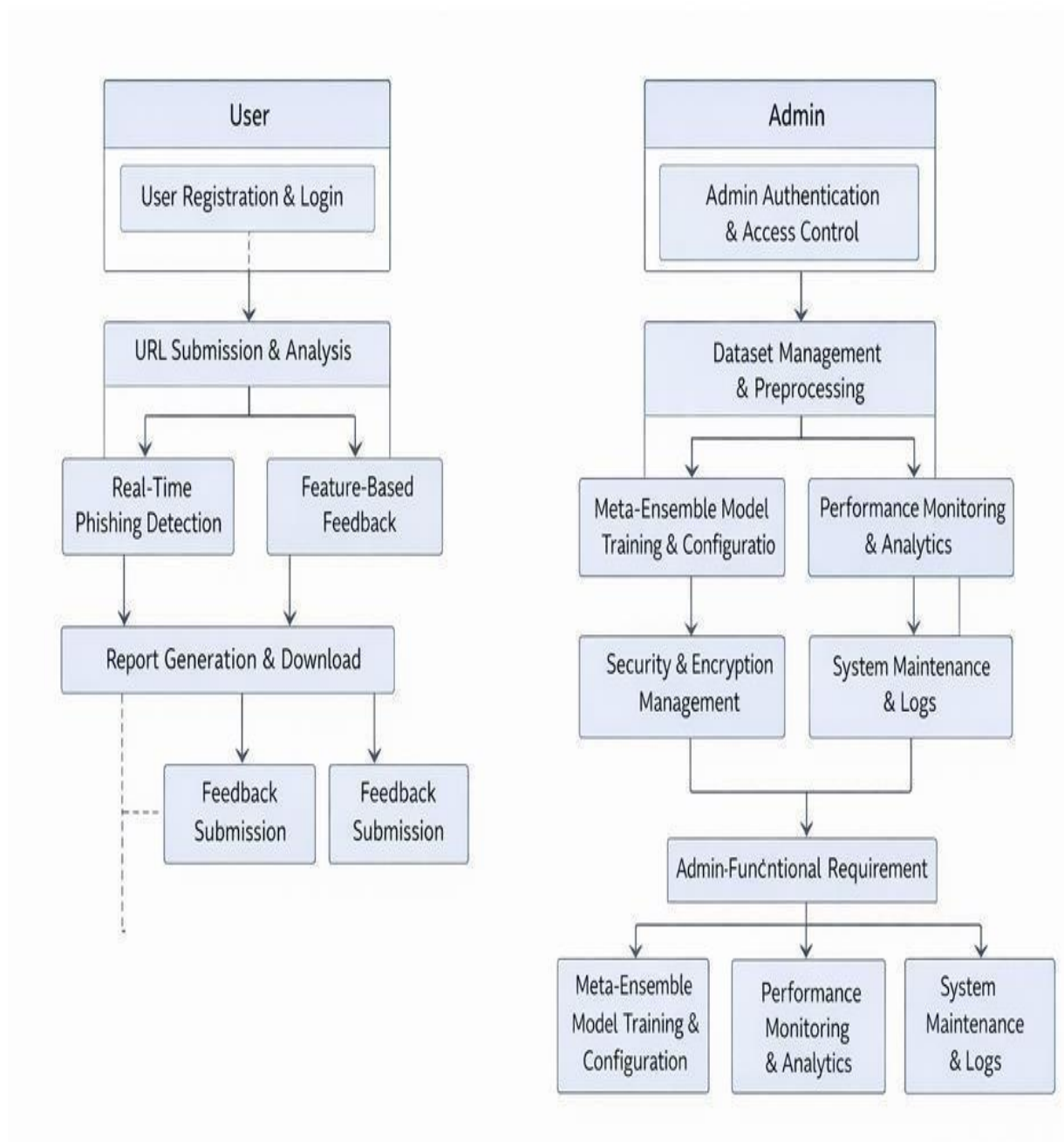
USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



CLASS DIAGRAM

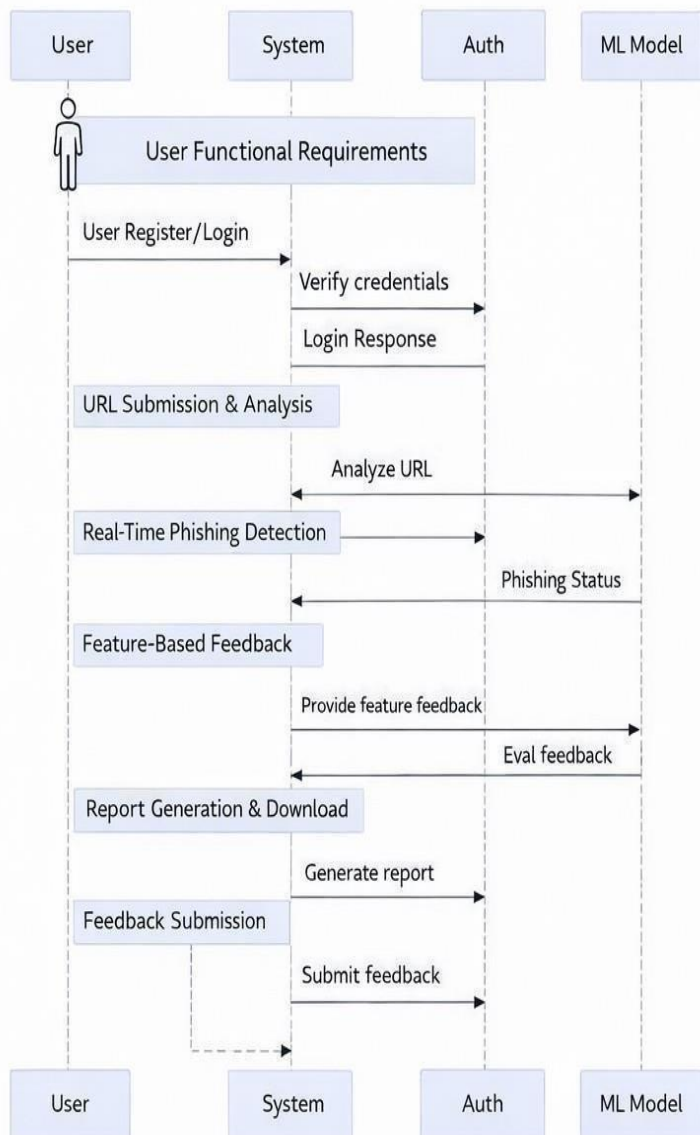
In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



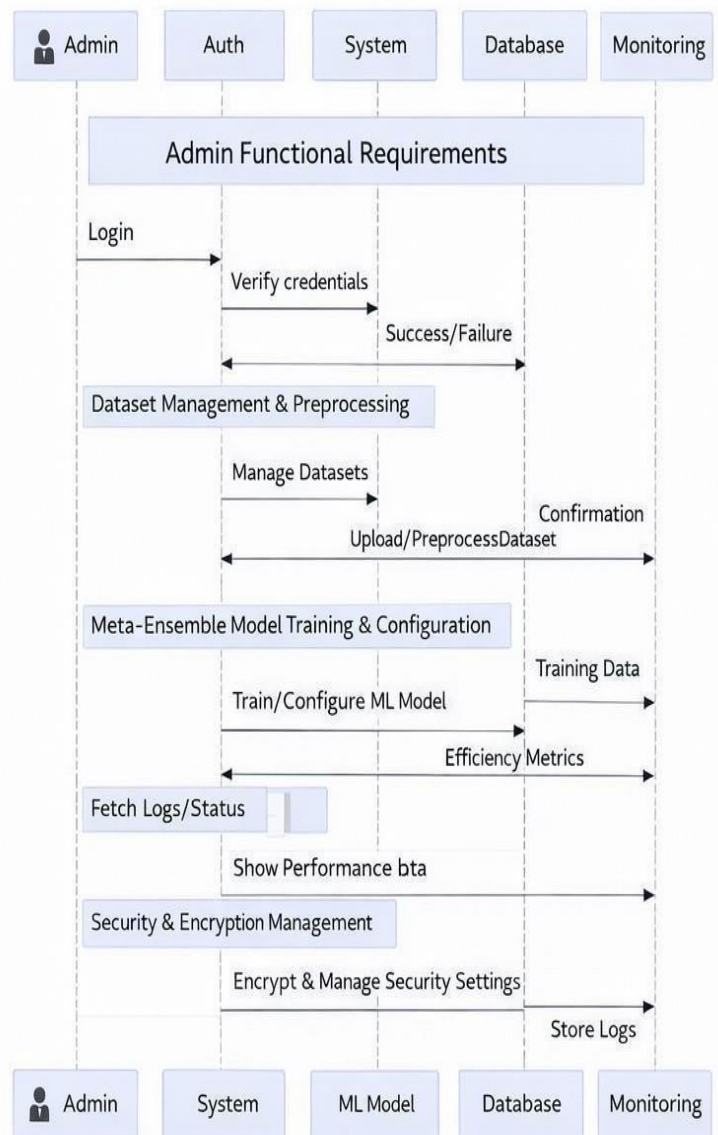
SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

User Functional Requirements

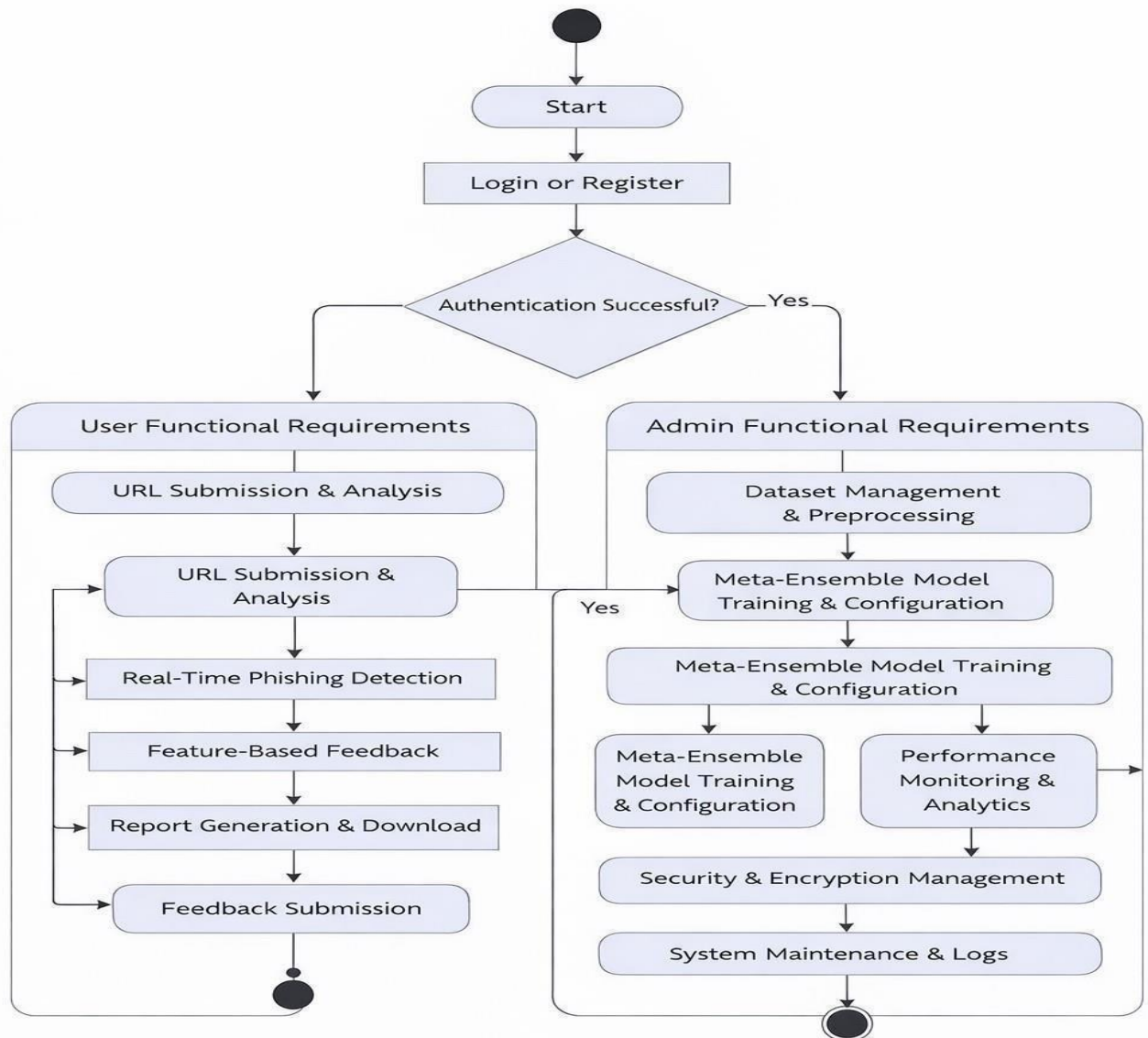


Admin Functional Requirements

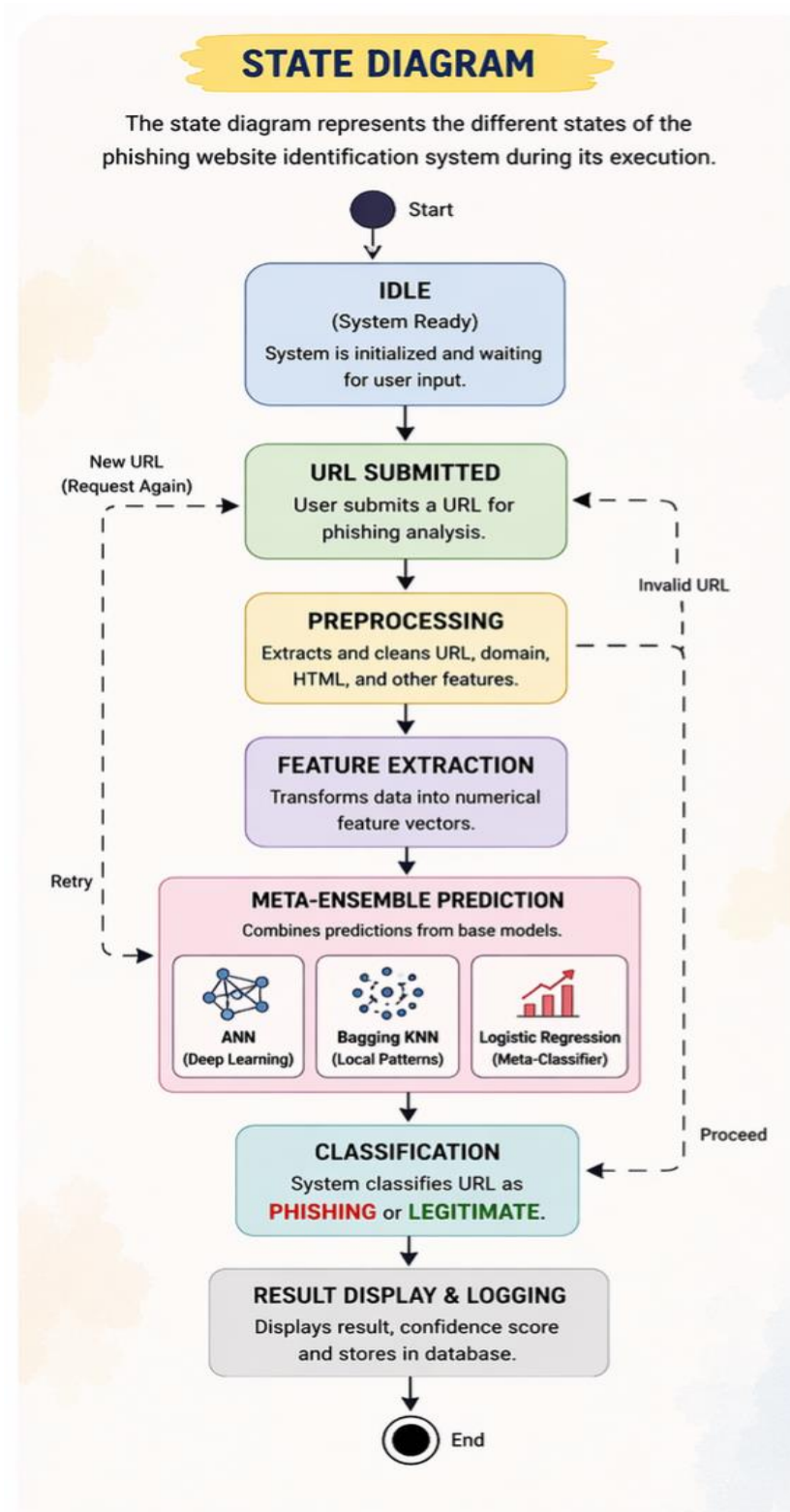


ACTIVITY DIAGRAM

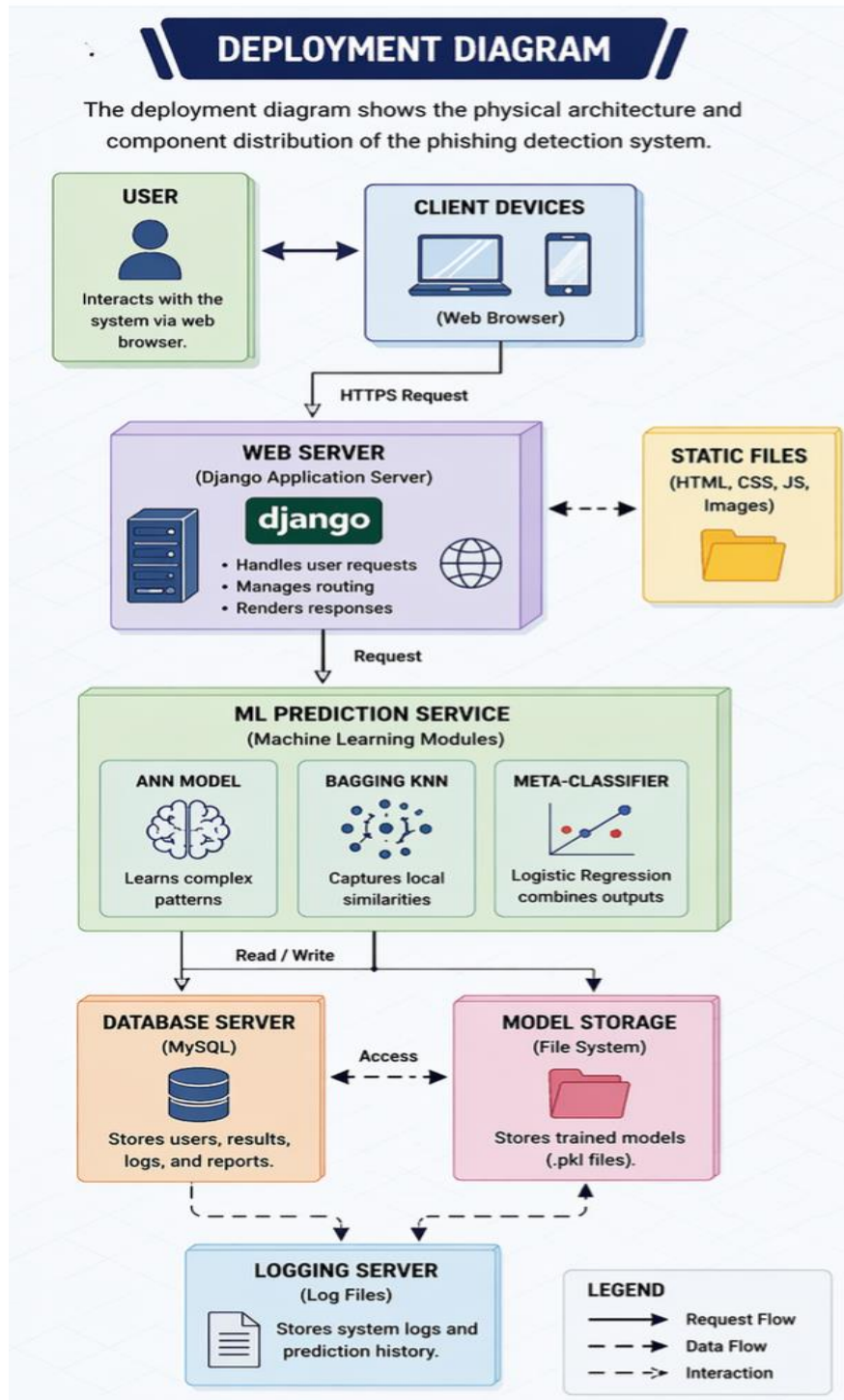
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



5.4 STATE DIAGRAM



5.5 DEPLOYMENT DIAGRAM



6. SYSTEM STUDY

FEASIBILITY ANALYSIS

Feasibility analysis is used to determine whether the proposed system is practical, cost-effective, and beneficial. It evaluates different aspects such as economic, technical, and social feasibility to ensure that the system can be successfully developed and implemented.

6.1 ECONOMIC FEASIBILITY

Economic feasibility determines whether the system is financially viable. The proposed phishing detection system is cost-effective as it uses open-source technologies such as Python, Django, and machine learning libraries like TensorFlow and Scikit-learn. These tools are freely available, which reduces software costs. The hardware requirements are minimal and only require a standard computer with moderate specifications. There is no need for expensive infrastructure. Publicly available datasets are used, eliminating the need for purchasing costly data. Development and maintenance costs are also low due to the modular design of the system. The system helps prevent financial losses caused by phishing attacks and reduces manual effort, making it economically beneficial.

6.2 TECHNICAL FEASIBILITY

Technical feasibility evaluates whether the required technology and resources are available for system development. The system is developed using Python, Django, HTML, CSS, and JavaScript, which are widely used and reliable technologies. Machine learning models such as Artificial Neural Networks, K-Nearest Neighbour, and Logistic Regression are used for phishing detection. These models are implemented using standard libraries like TensorFlow and Scikit-learn, ensuring efficient and accurate performance. The system supports deployment on local servers as well as cloud platforms. It is scalable and can be upgraded easily without major technical changes. The modular structure of the system allows easy maintenance and integration with other applications.

6.3 SOCIAL FEASIBILITY

Social feasibility focuses on user acceptance and the impact of the system on society. The proposed system helps users identify phishing websites and avoid cyber threats, improving online safety. It provides a simple and user-friendly interface where users can enter a URL and receive results easily. This makes the system suitable for both technical and non-technical users.

The system benefits organizations by enhancing security and reducing cyber fraud. It ensures data privacy and builds trust among users. It also creates awareness about phishing attacks and promotes safe internet usage.

7. IMPLEMENTATION

7.1 Modules used in project

Service Provider Module

The service provider acts as the administrator of the system. This module is responsible for managing the overall system activities, monitoring performance, uploading datasets, training machine learning models, and viewing detection results. It controls all operations and ensures the system works efficiently.

Remote User Module

This module allows users to access the system remotely. Users can register, log in, and submit website URLs to check whether they are phishing or legitimate. It provides a simple interface for user interaction.

View and Authorize Users Module

In this module, the service provider can view all registered users and authorize them to access the system. Only authorized users are allowed to use the phishing detection features, ensuring security and controlled access.

Dataset Upload Module

This module enables the service provider to upload phishing and legitimate datasets into the system. These datasets are used for training and testing the machine learning models.

Train and Test Module

This module is used to train machine learning models using the uploaded dataset and test their performance. It helps in improving the accuracy of phishing detection.

Phishing Detection Module

This module processes the user-entered URL and predicts whether it is phishing or legitimate using trained models. It displays the result to the user.

View Results Module

This module allows both users and the service provider to view the detection results. It helps in analyzing system performance and user activity.

Database Module

This module manages the storage of user details, datasets, trained models, and detection history. It ensures data is stored securely and can be retrieved when needed.

7.2 Procedure

- System Initialization
 - Start the application
 - Load system modules and database
 - Display login and registration page
- User Authentication
 - User registers with required details
 - Service provider authorizes the user
 - User logs into the system
- URL Input
 - User enters the website URL
 - System validates the input format
 - URL is sent for processing
- Feature Extraction
 - Extract URL-based features
 - Analyze domain and structure details
 - Identify special characters and patterns
- Data Preprocessing
 - Clean and organize data
 - Handle missing values
 - Normalize the data for model input

- Model Analysis
 - Input is given to trained ML models
 - Models used: ANN, KNN, Logistic Regression
 - Each model processes the data
- Prediction
 - Meta-ensemble combines model outputs
 - Final result is generated
 - URL is classified as phishing or legitimate
- Result Display
 - Show prediction result to user
 - Provide clear output status
- Data Storage
 - Store results in database
 - Maintain user activity history
- System Management
 - Service provider monitors system
 - Updates datasets and retrains models
 - Manages users and system performance

8. Source Code

```
from django.shortcuts import render, redirect
from django.contrib.auth import login, authenticate, logout
from django.contrib.auth.decorators import login_required
from django.contrib.auth.models import User
from django.contrib import messages
from django.http import JsonResponse
from django.contrib.auth.models import UserProfile

def register_view(request):
    """User registration"""
    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        password2 = request.POST.get('password2')
        phone = request.POST.get('phone', '')

        # validation
        if password != password2:
            messages.error(request, 'Passwords do not match')
            return redirect('accounts:register')

        if User.objects.filter(username=username).exists():
            messages.error(request, 'Username already exists')
            return redirect('accounts:register')

        if User.objects.filter(email=email).exists():
            messages.error(request, 'Email already registered')
            return redirect('accounts:register')

        # create user
        user = User.objects.create_user(
            username=username, email=email,
            password=password
        )

        # create profile
        UserProfile.objects.create(
            user=user, phone=phone
        )

        messages.success(request, 'Registration successful! Please login.')
        return redirect('accounts:login')
```

```

return render(request, 'accounts/register.html')

def login_view(request):
    """User login"""
    if request.user.is_authenticated:
        return redirect('detection:dashboard')

    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(request, username=username, password=password) if
        user is not None:
            login(request, user)
            messages.success(request, f'Welcome back, {user.username}!')

            # redirect based on user type
            if user.is_staff or user.is_superuser:
                return redirect('admin_panel:dashboard') else:
                return redirect('detection:dashboard')
            else:
                messages.error(request, 'Invalid username or password') return
                redirect('accounts:login')

    return render(request, 'accounts/login.html')

@login_required
def logout_view(request):
    """User logout"""
    logout(request)
    messages.success(request, 'Logged out successfully')
    return redirect('accounts:login')

@login_required
def profile_view(request):
    """View and update user profile"""
    profile, created = UserProfile.objects.get_or_create(user=request.user)

    if request.method == 'POST': #
        update user info
        request.user.first_name = request.POST.get('first_name', '')
        request.user.last_name = request.POST.get('last_name', '')
        request.user.email = request.POST.get('email', '') request.user.save()

        # update profile
        profile.phone = request.POST.get('phone', '')

```

```

profile.address = request.POST.get('address', '')
profile.email_notifications = request.POST.get('email_notifications') == 'on'
profile.save()

messages.success(request, 'Profile updated successfully')
return redirect('accounts:profile')

context = {
    'user': request.user, 'profile':
    profile
}

return render(request, 'accounts/profile.html', context)

import json
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required, user_passes_test
from django.contrib import messages
from django.http import JsonResponse
from django.conf import settings
from django.core.paginator import Paginator
from django.db.models import Count
from .models import Dataset, ModelTraining, SystemLog
from detection.models import DetectionHistory
from django.contrib.auth.models import User
import os
import pandas as pd
import sys
from datetime import datetime, timedelta

sys.path.append(str(settings.BASE_DIR / 'ml_models'))
from phishing_detector import HybridPhishingDetector

def is_admin(user):
    """Check if user is admin"""
    return user.is_staff or user.is_superuser

@login_required
@user_passes_test(is_admin)
def admin_dashboard(request):
    """Admin dashboard"""

    total_users = User.objects.count()
    total_detections = DetectionHistory.objects.count()
    phishing_detected = DetectionHistory.objects.filter(result='phishing').count()
    legitimate_detected = DetectionHistory.objects.filter(result='legitimate').count()
    total_datasets = Dataset.objects.count()

    recent_logs = SystemLog.objects.all()[:10]
    recent_trainings = ModelTraining.objects.all()[:5]

    today = datetime.now().date()
    dates = [(today - timedelta(days=i)).strftime('%Y-%m-%d') for i in range(6, -1, -1)]

```

```

detection_counts = []

for date_str in dates:
    date_obj = datetime.strptime(date_str, '%Y-%m-%d').date()
    count = DetectionHistory.objects.filter(
        created_at__date=date_obj
    ).count()
    detection_counts.append(count)

context = {
    'total_users': total_users,
    'total_detections': total_detections,
    'phishing_detected': phishing_detected,
    'legitimate_detected': legitimate_detected,
    'total_datasets': total_datasets,
    'recent_logs': recent_logs,
    'recent_trainings': recent_trainings,
    'chart_dates': json.dumps(dates),
    'chart_counts': json.dumps(detection_counts),
}

return render(request, 'admin_panel/dashboard.html', context)

@login_required
@user_passes_test(is_admin)
def manage_users(request):
    """Manage users"""

    users = User.objects.all().order_by('-date_joined')

    paginator = Paginator(users, 15)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    context = {
        'page_obj': page_obj,
    }

    return render(request, 'admin_panel/manage_users.html', context)

@login_required
@user_passes_test(is_admin)
def toggle_user_status(request, user_id):
    """Activate/deactivate user"""

    user = get_object_or_404(User, id=user_id)
    user.is_active = not user.is_active
    user.save()

    status = 'activated' if user.is_active else 'deactivated'
    messages.success(request, f'User {user.username} {status}')

```

```

SystemLog.objects.create(
    log_type='info',
    message=f'User {user.username} {status} by {request.user.username}',
    user=request.user,
    ip_address=request.META.get('REMOTE_ADDR')
)

return redirect('admin_panel:manage_users')


@login_required
@user_passes_test(is_admin)
def manage_datasets(request):
    """Manage datasets"""

    datasets = Dataset.objects.all()

    context = {
        'datasets': datasets,
    }

    return render(request, 'admin_panel/manage_datasets.html', context)


@login_required
@user_passes_test(is_admin)
def upload_dataset(request):
    """Upload new dataset"""

    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description')
        file = request.FILES.get('file')

        if not file:
            messages.error(request, 'Please select a file')
            return redirect('admin_panel:upload_dataset')

        try:
            df = pd.read_csv(file)

            if 'label' not in df.columns:
                messages.error(request, 'Dataset must contain a "label" column')
                return redirect('admin_panel:upload_dataset')

            total_samples = len(df)
            phishing_samples = len(df[df['label'] == 1])
            legitimate_samples = len(df[df['label'] == 0])

            dataset = Dataset.objects.create(
                name=name,
                description=description,
                file_path=file,
                total_samples=total_samples,

```

```

        phishing_samples=phishing_samples,
        legitimate_samples=legitimate_samples,
        uploaded_by=request.user
    )

    messages.success(request, f'Dataset "{name}" uploaded successfully')

    SystemLog.objects.create(
        log_type='success',
        message=f'Dataset "{name}" uploaded by {request.user.username}',
        user=request.user,
        ip_address=request.META.get('REMOTE_ADDR')
    )

    return redirect('admin_panel:manage_datasets')

except Exception as e:
    messages.error(request, f'Error uploading dataset: {str(e)}')
    return redirect('admin_panel:upload_dataset')

return render(request, 'admin_panel/upload_dataset.html')

@login_required
@user_passes_test(is_admin)
def train_model(request, dataset_id):
    """Train ML model with dataset"""

    dataset = get_object_or_404(Dataset, id=dataset_id)

    try:
        training = ModelTraining.objects.create(
            dataset=dataset,
            model_name='Hybrid Stacked Ensemble',
            status='training',
            started_by=request.user
        )

        detector = HybridPhishingDetector()

        file_path = dataset.file_path.path
        history = detector.train_full_model(file_path, apply_smote=True, k_folds=5)

        model_dir = settings.ML_MODELS_DIR / 'trained'
        detector.save_models(str(model_dir))

        training.status = 'completed'
        training.accuracy = history['stacked_metrics']['accuracy']
        training.precision = history['stacked_metrics']['precision']
        training.recall = history['stacked_metrics']['recall']
        training.f1_score = history['stacked_metrics']['f1_score']
        training.model_file_path = str(model_dir)
        training.completed_at = datetime.now()
        training.parameters = history

```

```

        training.save()

        messages.success(request, f'Model trained successfully! Accuracy: {training.accuracy:.4f}')

        SystemLog.objects.create(
            log_type='success',
            message=f'Model trained with dataset "{dataset.name}" by {request.user.username}',
            user=request.user,
            ip_address=request.META.get('REMOTE_ADDR')
        )

    except Exception as e:
        training.status = 'failed'
        training.error_message = str(e)
        training.save()

        messages.error(request, f'Training failed: {str(e)}')

        SystemLog.objects.create(
            log_type='error',
            message=f'Model training failed: {str(e)}',
            user=request.user,
            ip_address=request.META.get('REMOTE_ADDR')
        )

    return redirect('admin_panel:model_performance')

@login_required
@user_passes_test(is_admin)
def model_performance(request):
    """View model performance metrics"""

    trainings = ModelTraining.objects.all()

    context = {
        'trainings': trainings,
    }

    return render(request, 'admin_panel/model_performance.html', context)

@login_required
@user_passes_test(is_admin)
def system_logs(request):
    """View system logs"""

    logs = SystemLog.objects.all()

    log_type = request.GET.get('type', 'all')
    if log_type != 'all':
        logs = logs.filter(log_type=log_type)

    paginator = Paginator(logs, 20)

```

```

page_number = request.GET.get('page')
page_obj = paginator.get_page(page_number)

context = {
    'page_obj': page_obj,
    'log_type': log_type,
}

return render(request, 'admin_panel/system_logs.html', context)
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import JsonResponse, HttpResponse
from django.conf import settings
from django.core.paginator import Paginator
from .models import DetectionHistory, PhishingFeature
from .utils import extract_url_features, extract_email_features, extract_features, detect_input_type,
get_feature_importance_explanation, is_valid_email, is_valid_url
import joblib
import json
import os
from datetime import datetime
import sys
import logging

logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s -
%(message)s')

sys.path.append(str(settings.BASE_DIR / 'ml_models'))
from phishing_detector import HybridPhishingDetector

detector = None

def load_detector():
    """Load ML model"""
    global detector
    if detector is None:
        try:
            detector = HybridPhishingDetector()
            model_path = settings.ML_MODELS_DIR / 'trained'
            detector.load_models(str(model_path))
            print("✓ ML Model loaded successfully")
        except Exception as e:
            print(f"✗ Error loading ML model: {e}")
            detector = None
    return detector

@login_required
def dashboard_view(request):
    """User dashboard"""

    total_scans = DetectionHistory.objects.filter(user=request.user).count()

```

```

phishing_detected = DetectionHistory.objects.filter(
    user=request.user,
    result='phishing'
).count()
legitimate_detected = DetectionHistory.objects.filter(
    user=request.user,
    result='legitimate'
).count()

recent_detections = DetectionHistory.objects.filter(
    user=request.user
)[:5]

context = {
    'total_scans': total_scans,
    'phishing_detected': phishing_detected,
    'legitimate_detected': legitimate_detected,
    'recent_detections': recent_detections,
}

return render(request, 'detection/dashboard.html', context)

@login_required
def analyze_view(request):
    """Analyze URL or email for phishing"""

    if request.method == 'POST':
        detection_type = request.POST.get('detection_type', 'auto')
        input_text = request.POST.get('input_text', "").strip()

        if not input_text:
            messages.error(request, 'Please enter a URL or email to analyze')
            return redirect('detection:analyze')

        try:
            if detection_type == 'auto':
                detected_type = detect_input_type(input_text)
            else:
                detected_type = detection_type

            if detected_type == 'email' and not is_valid_email(input_text):
                messages.error(request, 'Please enter a valid email address')
                return redirect('detection:analyze')

            elif detected_type == 'url' and not is_valid_url(input_text):
                if not (input_text.startswith('www.') or '.' in input_text):
                    messages.error(request, 'Please enter a valid URL')
                    return redirect('detection:analyze')

            if detected_type == 'email':
                features = extract_email_features(input_text)
            else:
                features = extract_url_features(input_text)

```

```

detector = load_detector()
if detector is None:
    messages.error(request, 'ML Model not available. Please contact administrator.')
    return redirect('detection:analyze')

result = detector.predict(features)
logger.info(f'Prediction Result: {result['prediction']}, Confidence: {result['confidence']:.4f}')

confidence_percentage = result['confidence'] * 100

important_features = get_feature_importance_explanation(features)

detection = DetectionHistory.objects.create(
    user=request.user,
    detection_type=detected_type,
    input_text=input_text,
    result=result['prediction'],
    confidence_score=confidence_percentage,
    features_used=features
)

logger.info(f'Detection saved: ID={detection.id}, Result={detection.result}')

for feature_name, feature_value, score in important_features:
    PhishingFeature.objects.create(
        detection=detection,
        feature_name=feature_name,
        feature_value=feature_value,
        importance=score
    )

return redirect('detection:result', detection_id=detection.id)

except Exception as e:
    logger.error(f'Error during analysis: {str(e)}", exc_info=True)
    messages.error(request, f'Error during analysis: {str(e)}')
    return redirect('detection:analyze')

return render(request, 'detection/analyze.html')

@login_required
def result_view(request, detection_id):
    """View detection result"""

    detection = get_object_or_404(
        DetectionHistory,
        id=detection_id,
        user=request.user
    )

    features = detection.features.all()

```

```

context = {
    'detection': detection,
    'features': features,
}

return render(request, 'detection/result.html', context)

@login_required
def history_view(request):
    """View detection history"""

    detections = DetectionHistory.objects.filter(user=request.user)

    filter_type = request.GET.get('type', 'all')
    if filter_type != 'all':
        detections = detections.filter(detection_type=filter_type)

    filter_result = request.GET.get('result', 'all')
    if filter_result != 'all':
        detections = detections.filter(result=filter_result)

    paginator = Paginator(detections, 10)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    context = {
        'page_obj': page_obj,
        'filter_type': filter_type,
        'filter_result': filter_result,
    }

    return render(request, 'detection/history.html', context)

@login_required
def delete_detection(request, detection_id):
    """Delete detection record"""

    detection = get_object_or_404(
        DetectionHistory,
        id=detection_id,
        user=request.user
    )

    detection.delete()
    messages.success(request, 'Detection record deleted')
    return redirect('detection:history')

@login_required
def download_report(request, detection_id):
    """Download detection report as CSV"""

```

```

detection = get_object_or_404(
    DetectionHistory,
    id=detection_id,
    user=request.user
)

response = HttpResponse(content_type='text/csv')
response['Content-Disposition'] = f'attachment; filename="detection_report_{detection.id}.csv"'

import csv
writer = csv.writer(response)

writer.writerow(['Phishing Detection Report'])
writer.writerow([])
writer.writerow(['Field', 'Value'])
writer.writerow(['Detection ID', detection.id])
writer.writerow(['Type', detection.get_detection_type_display()])
writer.writerow(['Input', detection.input_text])
writer.writerow(['Result', detection.get_result_display()])
writer.writerow(['Confidence', f'{detection.confidence_score:.2f}%'])
writer.writerow(['Date', detection.created_at.strftime('%Y-%m-%d %H:%M:%S')])
writer.writerow([])

writer.writerow(['Important Features'])
writer.writerow(['Feature Name', 'Value', 'Importance'])

for feature in detection.features.all():
    writer.writerow([feature.feature_name, feature.feature_value, feature.importance])

return response
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse
from django.conf import settings
from detection.models import DetectionHistory
from .models import Report
import csv
from datetime import datetime

@login_required
def generate_report(request):
    """Generate user activity report"""

    if request.method == 'POST':
        report_type = request.POST.get('report_type')

        response = HttpResponse(content_type='text/csv')
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        response['Content-Disposition'] = f'attachment; filename="report_{timestamp}.csv"'

        writer = csv.writer(response)

        if report_type == 'detection':

```

```

detections = DetectionHistory.objects.filter(user=request.user)

writer.writerow(['Phishing Detection History Report'])
writer.writerow(['Generated:', datetime.now().strftime('%Y-%m-%d %H:%M:%S')])
writer.writerow(['User:', request.user.username])
writer.writerow([])

writer.writerow(['ID', 'Type', 'Input', 'Result', 'Confidence', 'Date'])

for detection in detections:
    writer.writerow([
        detection.id,
        detection.get_detection_type_display(),
        detection.input_text,
        detection.get_result_display(),
        f'{detection.confidence_score * 100:.2f}%',
        detection.created_at.strftime('%Y-%m-%d %H:%M:%S')
    ])

return response

return render(request, 'reports/generate_report.html')

```

9. SYSTEM TESTING

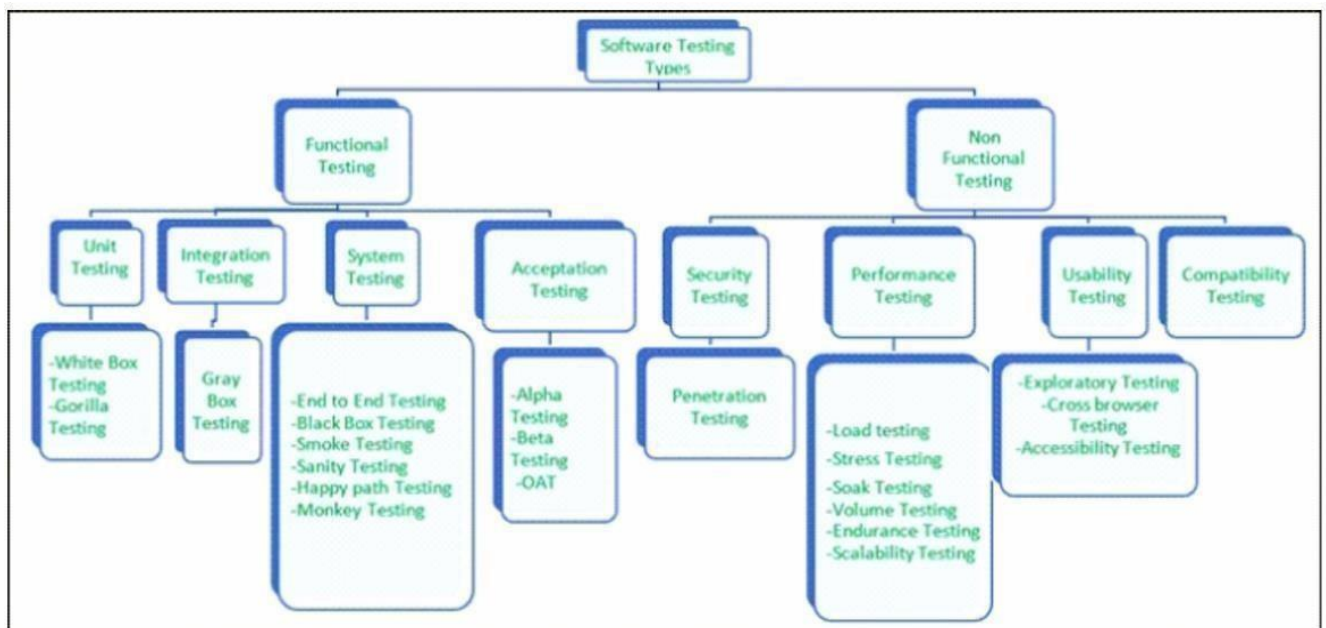
9.1 Types of Testing

System Testing

Types of Software Testing: Different Testing Types with Details

We, as testers, are aware of the various types of Software Testing like Functional Testing, Non- Functional Testing, Automation Testing, Agile Testing, and their sub-types, etc. Each type of testing has its own features, advantages, and disadvantages as well. However, in this tutorial, we have covered mostly each and every type of software testing which we usually use in our day-to-day testing life.

Different Types of Software Testing



Functional Testing

There are four main types of functional testing.

#1) Unit Testing

Unit testing is a type of software testing which is done on an individual unit or component to test

its corrections. Typically, Unit testing is done by the developer at the application development phase. Each unit in unit testing can be viewed as a method, function, procedure, or object. Developers often use test automation tools such as NUnit, Xunit, JUnit for the test execution. Unit testing is important because we can find more defects at the unit test level. For example, there is a simple calculator application. The developer can write the unit test to check if the user can enter two numbers and get the correct sum for addition functionality.

a) White Box Testing

White box testing is a test technique in which the internal structure or code of an application is visible and accessible to the tester. In this technique, it is easy to find loopholes in the design of an application or fault in business logic. Statement coverage and decision coverage/branch coverage are examples of white box test techniques.

b) Gorilla Testing

Gorilla testing is a test technique in which the tester and/or developer test the module of the application thoroughly in all aspects. Gorilla testing is done to check how robust your application is.

For example, the tester is testing the pet insurance company's website, which provides the service of buying an insurance policy, tag for the pet, Lifetime membership. The tester can focus on any one module, let's say, the insurance policy module, and test it thoroughly with positive and negative test scenarios.

#2) Integration Testing

Integration testing is a type of software testing where two or more modules of an application are logically grouped together and tested as a whole. The focus of this type of testing is to find the defect on interface, communication, and data flow among modules. Top-down or Bottom-up approach is used while integrating modules into the whole system.

This type of testing is done on integrating modules of a system or between systems. For example, a user is buying a flight ticket from any airline website. Users can see flight details and payment information while buying a ticket, but flight details and payment processing are two different systems. Integration testing should be done while integrating of airline website and payment processing system.

a) Gray box testing

As the name suggests, gray box testing is a combination of white-box testing and black-box testing. Testers have partial knowledge of the internal structure or code of an application.

#3) System Testing

System testing is types of testing where tester evaluates the whole system against the specified requirements.

a)End to End Testing

It involves testing a complete application environment in a situation that mimics real-world use, such as interacting with a database, using network communications, or interacting with other hardware, applications, or systems if appropriate.

For example, a tester is testing a pet insurance website. End to End testing involves testing of buying an insurance policy, LPM, tag, adding another pet, updating credit card information on users' accounts, updating user address information, receiving order confirmation emails and policy documents.

b)Black Box Testing

Blackbox testing is a software testing technique in which testing is performed without knowing the internal structure, design, or code of a system under test. Testers should focus only on the input and output of test objects. Detailed information about the advantages, disadvantages, and types of Black Box testing can be found [here](#).

c)Smoke Testing

Smoke testing is performed to verify that basic and critical functionality of the system under test is working fine at a very high level. Whenever a new build is provided by the development team, then the Software Testing team validates the build and ensures that no major issue exists. The testing team will ensure that the build is stable, and a detailed level of testing will be carried out further. For example, tester is testing pet insurance website. Buying an insurance policy, adding another pet, providing quotes are all basic and critical functionality of the application. Smoke testing for this website verifies that all these functionalities are working fine before doing any in-depth testing.

d) Sanity Testing

Sanity testing is performed on a system to verify that newly added functionality or bug fixes are working fine. Sanity testing is done on stable build. It is a subset of the regression test. For example, a tester is testing a pet insurance website. There is a change in the discount for buying a policy for second pet. Then sanity testing is only performed on buying insurance policy module.

e) Happy path Testing

The objective of Happy Path Testing is to test an application successfully on a positive flow. It does not look for negative or error conditions. The focus is only on valid and positive inputs through which the application generates the expected output.

f) Monkey Testing

Monkey Testing is carried out by a tester, assuming that if the monkey uses the application, then how random input and values will be entered by the Monkey without any knowledge or understanding of the application.

The objective of Monkey Testing is to check if an application or system gets crashed by providing random input values/data. Monkey Testing is performed randomly, no test cases are scripted, and it is not necessary to be aware of the full functionality of the system.

#4) Acceptance Testing

Acceptance testing is a type of testing where client/business/customer test the software with real time business scenarios.

The client accepts the software only when all the features and functionalities work as expected. This is the last phase of testing, after which the software goes into production. This is also called User Acceptance Testing (UAT).

a) Alpha Testing

Alpha testing is a type of acceptance testing performed by the team in an organization to find as many defects as possible before releasing software to customers.

For example, the pet insurance website is under UAT. UAT team will run real-time scenarios like buying an insurance policy, buying annual membership, changing the address, ownership transfer of the pet in a same way the user uses the real website. The team can use test credit card information to process payment-related scenarios.

b)Beta Testing

Beta Testing is a type of software testing which is carried out by the clients/customers. It is performed in the Real Environment before releasing the product to the market for the actual end-users.

Beta Testing is carried out to ensure that there are no major failures in the software or product, and it satisfies the business requirements from an end-user perspective.

Beta Testing is successful when the customer accepts the software.

Usually, this testing is typically done by the end-users. This is the final testing done before releasing the application for commercial purposes. Usually, the Beta version of the software or product released is limited to a certain number of users in a specific area.

So, the end-user uses the software and shares the feedback with the company. The company then takes necessary action before releasing the software worldwide.

c)Operational acceptance testing (OAT)

Operational acceptance testing of the system is performed by operations or system administration staff in the production environment. The purpose of operational acceptance testing is to make sure that the system administrators can keep the system working properly for the users in a real-time environment.

The focus of the OAT is on the following points:

- Testing of backup and restore.
- Installing, uninstalling, upgrading software.
- The recovery process in case of natural disaster.
- User management.
- Maintenance of the software.

Non-Functional Testing

There are four main types of functional testing.

#1) Security Testing

It is a type of testing performed by a special team. Any hacking method can penetrate the system. Security Testing is done to check how the software, application, or website is secure from internal and/or external threats. This testing includes how much software is secure from malicious programs, viruses and how secure&strong the authorization and authentication processes are.

It also checks how software behaves for any hacker's attack&malicious programs and how software is maintained for data security after such a hacker attack.

a)Penetration Testing

Penetration Testing or Pen testing is the type of security testing performed as an authorized cyberattack on the system to find out the weak points of the system in terms of security.

Pen testing is performed by outside contractors, generally known as ethical hackers. That is why it is also known as ethical hacking. Contractors perform different operations like SQL injection, URL manipulation, Privilege Elevation, session expiry, and provide reports to the organization. Notes: Do not perform the Pen testing on your laptop/computer. Always take written permission to do pen tests.

#2) Performance Testing

Performance testing is testing of an application's stability and response time by applying load.

The word stability means the ability of the application to withstand in the presence of load. Response time is how quickly an application is available to users. Performance testing is done with the help of tools. Loader.IO, JMeter, LoadRunner, etc. are good tools available in the market.

Performance testing is testing of an application' s stability and response time by applying load.

The word stability means the ability of the application to withstand in the presence of load. Response time is how quickly an application is available to users. Performance testing is done with the help of tools. Loader.IO, JMeter, LoadRunner, etc. are good tools available in the market.

a)Load testing

Load testing is testing of an application' s stability and response time by applying load, which is equal to or less than the designed number of users for an application. For example, your application handles 100 users at a time with a response time of 3 seconds, then load testing can be done by applying a load of the maximum of 100 or less than 100 users. The goal is to verify that the application is responding within 3 seconds for all the users.

b)Stress Testing

Stress testing is testing an application' s stability and response time by applying load, which is more than the designed number of users for an application. For example, your application handles 1000 users at a time with a response time of 4 seconds, then stress testing can be done by applying a load of more than 1000 users. Test the application with 1100,1200,1300 users and notice the response time. The goal is to verify the stability of an application under stress.

c)Scalability Testing

Scalability testing is testing an application' s stability and response time by applying load, which is more than the designed number of users for an application. For example, your application handles 1000 users at a time with a response time of 2 seconds, then scalability testing can be done by applying a load of more than 1000 users and gradually increasing the number of users to find out where exactly my application is crashing. Let' s say my application is giving response time as follows:

- 1000 users -2 sec
- 1400 users -2 sec
- 4000 users -3 sec
- 5000 users -45 sec
- 5150 users- crash – This is the point that needs to identify in scalability testing

d) Volume testing (flood testing)

Volume testing is testing an application' s stability and response time by transferring a large volume of data to the database. Basic ally, it tests the capacity of the database to handle the data.

e) Endurance Testing (Soak Testing)

Endurance testing is testing an application' s stability and response time by applying load continuously for a longer period to verify that the application is working fine.

For example, car companies soak testing to verify that users can drive cars continuously for hours without any problem.

#3) Usability Testing

Usability testing is testing an application from the user' s perspective to check the look and feel and user-friendliness.

For example, there is a mobile app for stock trading, and a tester is performing usability testing. Testers can check the scenario like if the mobile app is easy to operate with one hand or not, scroll bar should be vertical, background colour of the app should be black and price of and stock is displayed in red or green colour.

The main idea of usability testing of this kind of app is that as soon as the user opens the app, the user should get a glance at the market.

a) Exploratory testing

Exploratory Testing is informal testing performed by the testing team. The objective of this testing is to explore the application and look for defects that exist in the application. Testers use the knowledge of the business domain to test the application. Test charters are used to guide the exploratory testing.

b)Cross browser testing

Cross browser testing is testing an application on different browsers, operating systems, mobile devices to see look and feel and performance. Why do we need cross-browser testing? The answer is different users use different operating systems, different browsers, and different mobile devices. The goal of the company is to get a good user experience regardless of those devices.

Browser stack provides all the versions of all the browsers and all mobile devices to test the application. For learning purposes, it is good to take the free trial given by browser stack for a few days.

c)Accessibility Testing

The aim of Accessibility Testing is to determine whether the software or application is accessible for disabled people or not.

Here, disability means deafness, colour blindness, mentally disabled, blind, old age, and other disabled groups. Various checks are performed, such as font size for visually disabled, colour and contrast for colour blindness, etc.

#4) Compatibility testing

This is a testing type in which it validates how software behaves and runs in a different environment, web servers, hardware, and network environment.

9.2 TEST CASES

Test Case ID	Module	Test Scenario	Input	Expected Output	Actual Output	Status
TC_01	User Authentication	Verify user login with valid credentials	Valid username & password	User successfully logged into dashboard	As Expected	Pass
TC_02	User Authentication	Verify login with invalid credentials	Invalid username or password	Error message: "Invalid Credentials"	As Expected	Pass
TC_03	URL Validation	Check system behavior with invalid URL format	abc123	System displays "Invalid URL Format"	As Expected	Pass
TC_04	URL Submission	Submit legitimate URL for detection	https://www.google.com	System classifies as Legitimate with high confidence	As Expected	Pass
TC_05	URL Submission	Submit known phishing URL	Suspicious phishing link	System classifies as Phishing with high confidence	As Expected	Pass
TC_06	Feature Extraction	Verify feature extraction	Valid URL	Extracted features	As Expected	Pass

Test Case ID	Module	Test Scenario	Input	Expected Output	Actual Output	Status
		process		(URL length, SSL status, domain age, etc.) displayed correctly		
TC_07	Model Prediction	Validate ensemble model prediction	URL input dataset	ANN + Bagging KNN predictions combined via Logistic Regression	As Expected	Pass
TC_08	Report Generation	Generate downloadable phishing report	Click "Download Report"	PDF/Report generated with prediction & confidence score	As Expected	Pass
TC_09	Admin Dataset Upload	Admin uploads new phishing dataset	CSV dataset file	<u>Dataset</u> successfully uploaded and stored in database	As Expected	Pass
TC_10	Performance	Check system	Multiple concurrent	Response	As	Pass

Test Case ID	Module	Test Scenario	Input	Expected Output	Actual Output	Status
	Test	response time under load	URL submissions	time < 5 seconds, no crash	Expected	

10. Outputs & screenshots

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/register/'. The page title is 'Create Account' with the subtitle 'Join PhishGuard Today'. The registration form includes the following fields: Username (with a 'Please fill out this field.' tooltip), Email, Phone (Optional), Password, and Confirm Password. A blue 'Register' button is at the bottom of the form, and a link 'Login here' is provided for existing users. The Windows taskbar at the bottom shows the date as 24-03-2026 and the time as 16:35.

WhatsApp x Register - Phishing Detection Syst x +

127.0.0.1:8000/register/ Summarize Chat

Import favorites For quick access, place your favorites here on the favorites bar. Manage favorites now Other favorites

Create Account

Join PhishGuard Today

Username Please fill out this field.

Email

Phone (Optional)

Password

Confirm Password

[Register](#)

Already have an account? [Login here](#)

High UV Now Search 16:35 24-03-2026

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/login/'. The page title is 'PhishGuard Login' with the subtitle 'Phishing Detection System'. The login form is centered and includes the following fields: Username (with a dropdown menu showing 'admin') and Password (with masked characters '*****'). A blue 'Login' button is at the bottom of the form, and a link 'Register here' is provided for new users. The Windows taskbar at the bottom shows the date as 24-03-2026 and the time as 16:29.

WhatsApp x Login - Phishing Detection System x +

127.0.0.1:8000/login/ Summarize Chat

Import favorites For quick access, place your favorites here on the favorites bar. Manage favorites now Other favorites

PhishGuard Login

Phishing Detection System

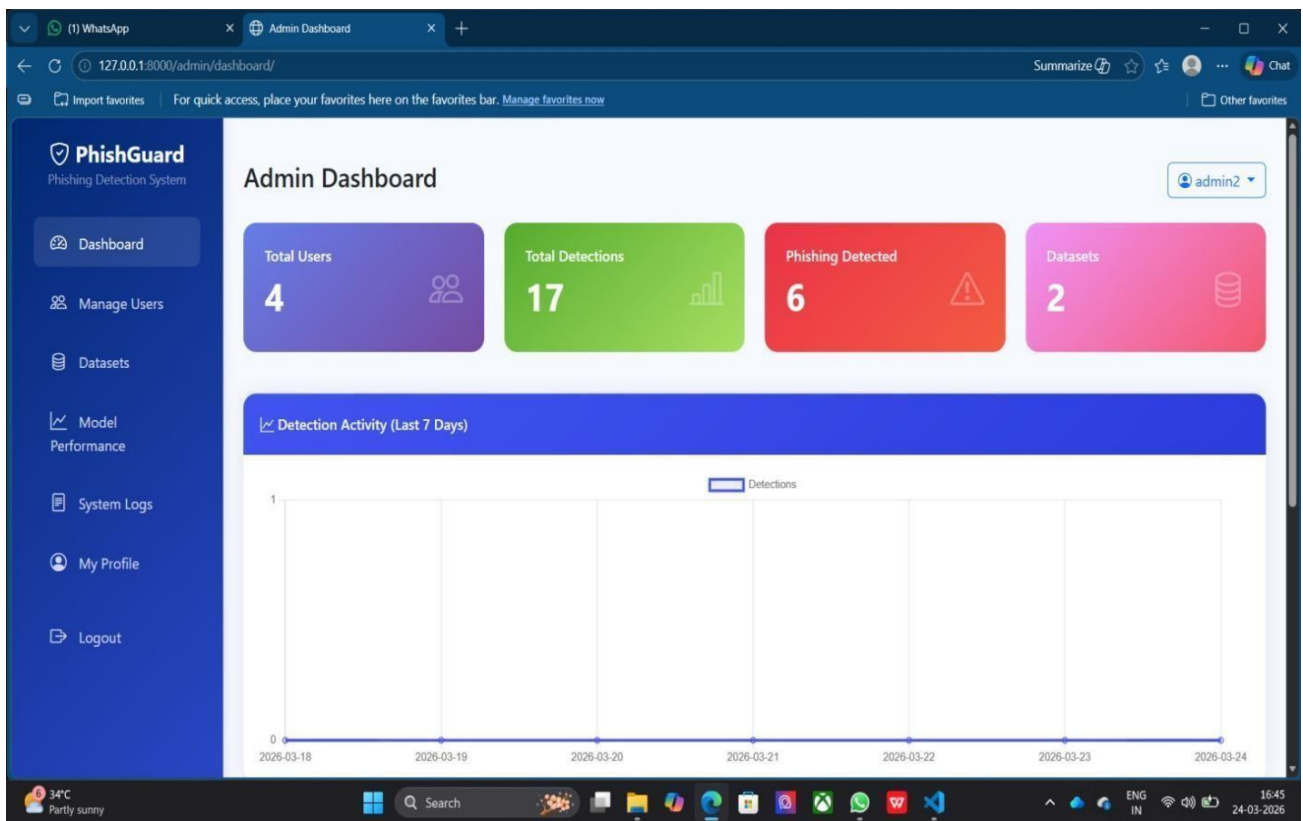
Username admin

Password *****

[Login](#)

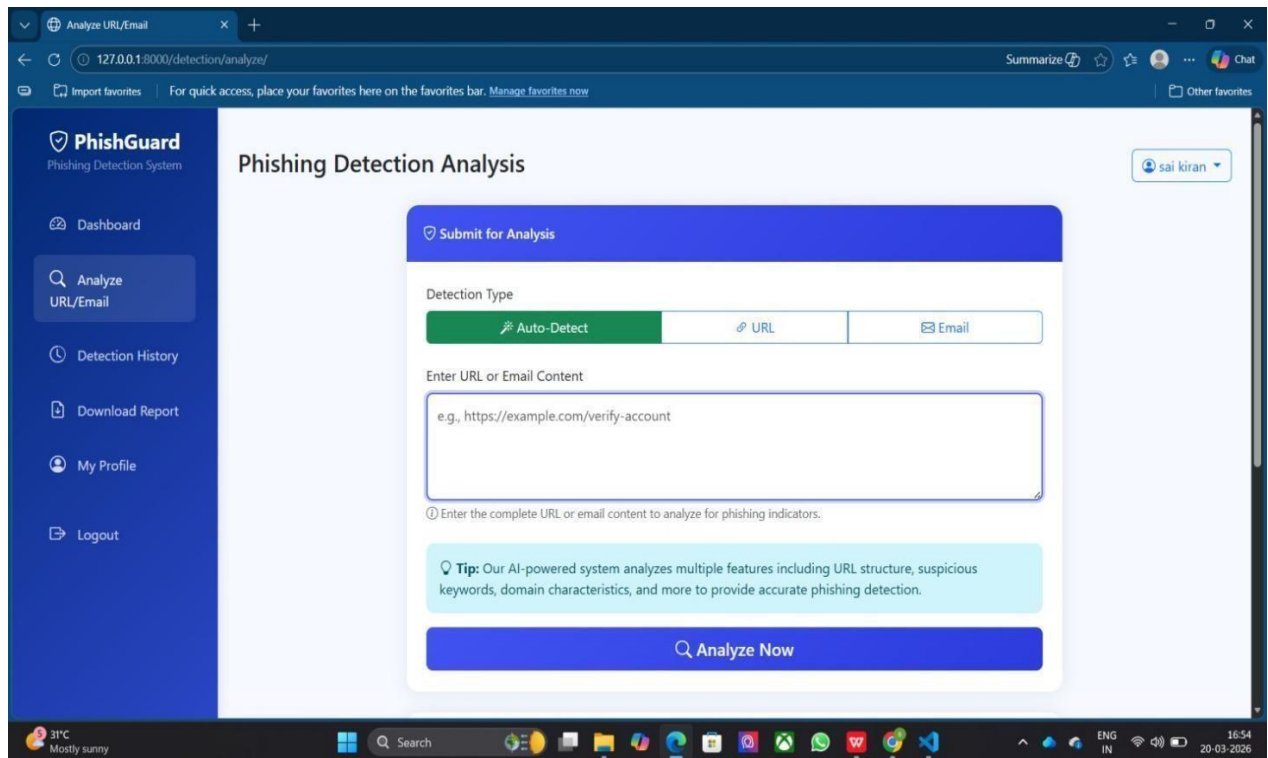
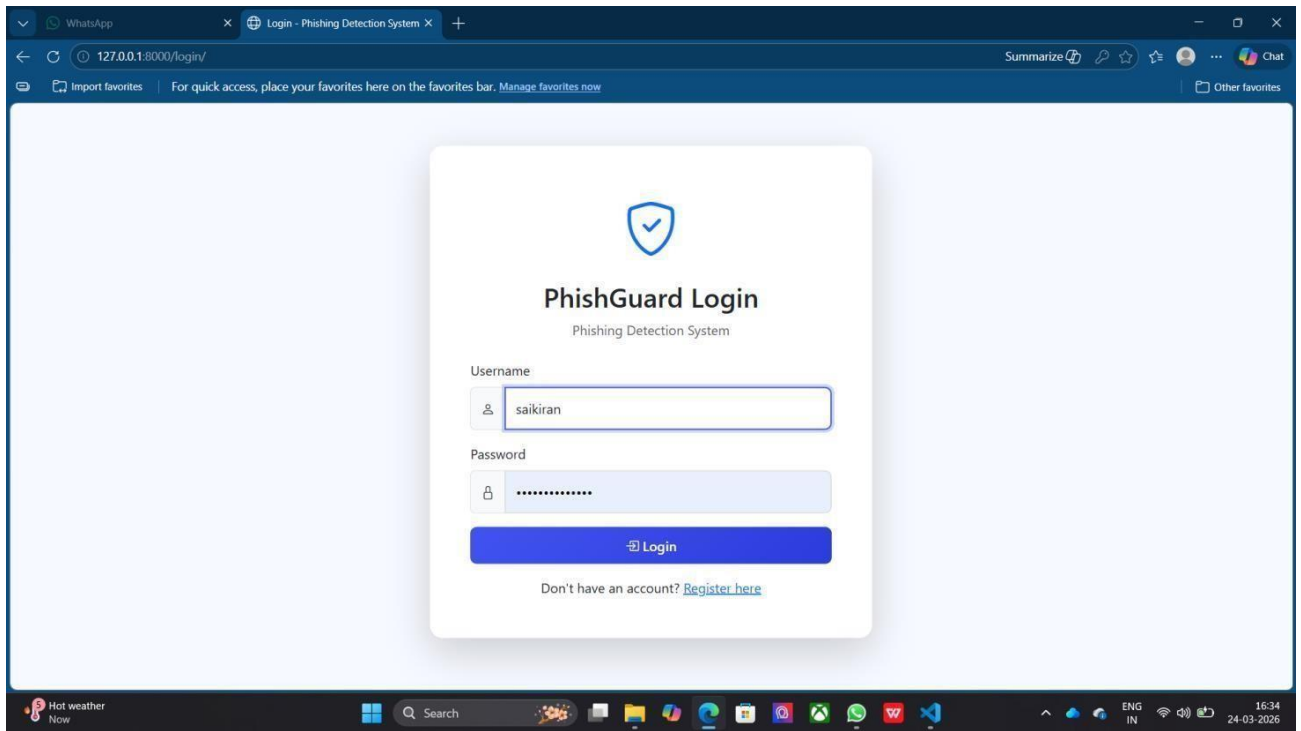
Don't have an account? [Register here](#)

34°C Partly sunny Search 16:29 24-03-2026



The screenshot displays the PhishGuard Dataset Management page. The left sidebar is the same as the dashboard. The main content area includes an 'Upload New Dataset' button and a table of 'Available Datasets'.

ID	Name	Description	Samples	Phishing	Legitimate	Uploaded By	Date	Status	Actions
#2	dataset	dataset	622	255	367	admin2	Feb 27, 2026	Active	<button>Train</button>
#1	phishing dataset	phishing data	10000	5000	5000	admin	Feb 23, 2026	Active	<button>Train</button>



WhatsApp x Dashboard x +

127.0.0.1:8000/detection/dashboard/

Import favorites For quick access, place your favorites here on the favorites bar. Manage favorites now

PhishGuard
Phishing Detection System

Dashboard

Analyze URL/Email

Detection History

Download Report

My Profile

Logout

127.0.0.1:8000/detection/analyze/

34°C
Partly sunny

Search

ENG IN

16:38
24-03-2026

Dashboard

sai kiran

Total Scans
10

Phishing Detected
2

Legitimate
8

Detection Rate
2%

Quick Actions

Analyze URL/Email View History

Recent Detections

Type	Input	Result	Confidence	Date	Action
URL	https://www.google-account-verify.com	Phishing	99.38%	Mar 24, 2026 16:37	View
URL	https://www.google.com	Legitimate	98.65%	Mar 24, 2026 16:37	View

WhatsApp x Detection Result x +

127.0.0.1:8000/detection/result/16/

Import favorites For quick access, place your favorites here on the favorites bar. Manage favorites now

PhishGuard
Phishing Detection System

Detection Result

admin

Detection Result #16

Input:

https://www.google-account-verify.com

PHISHING
This URL/Email is likely phishing!

Confidence Score
99.4%

Detection Type: URL

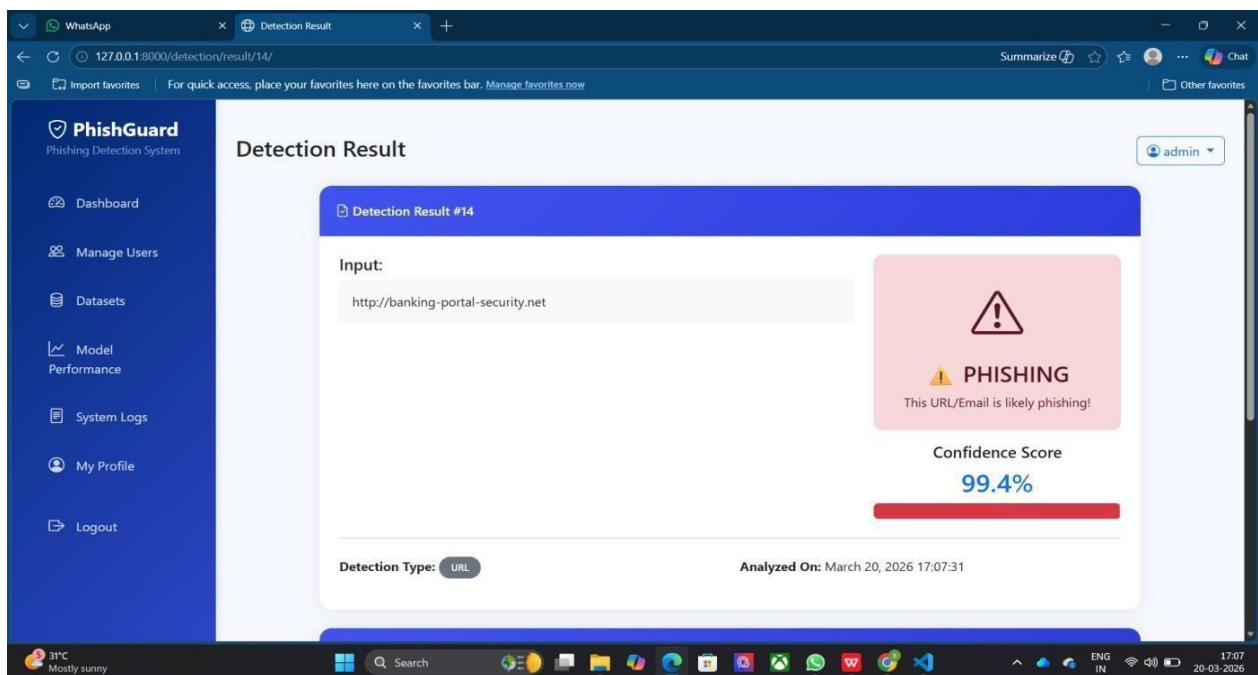
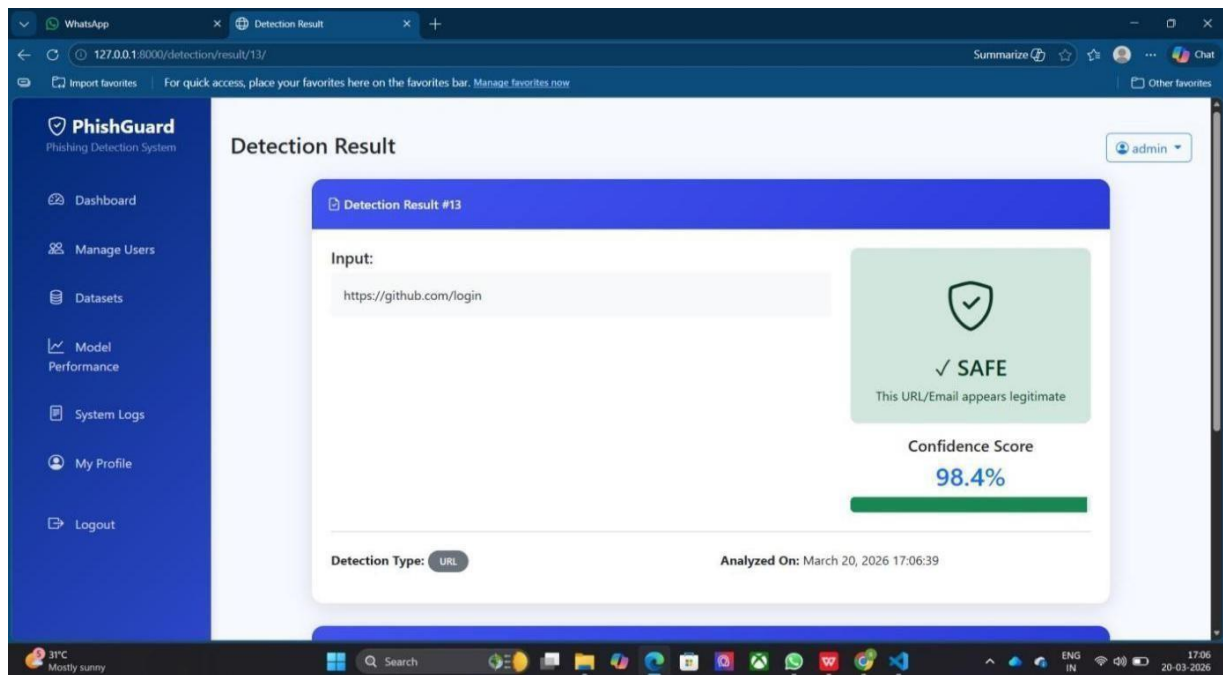
Analyzed On: March 20, 2026 17:13:43

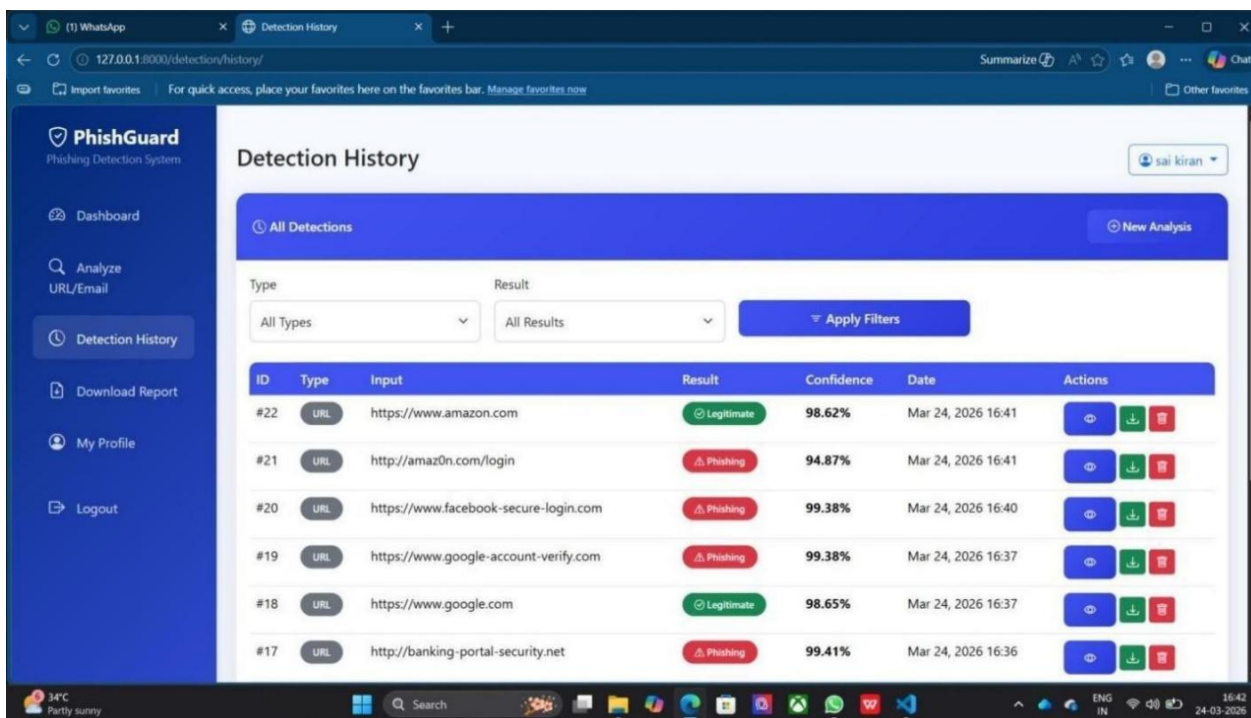
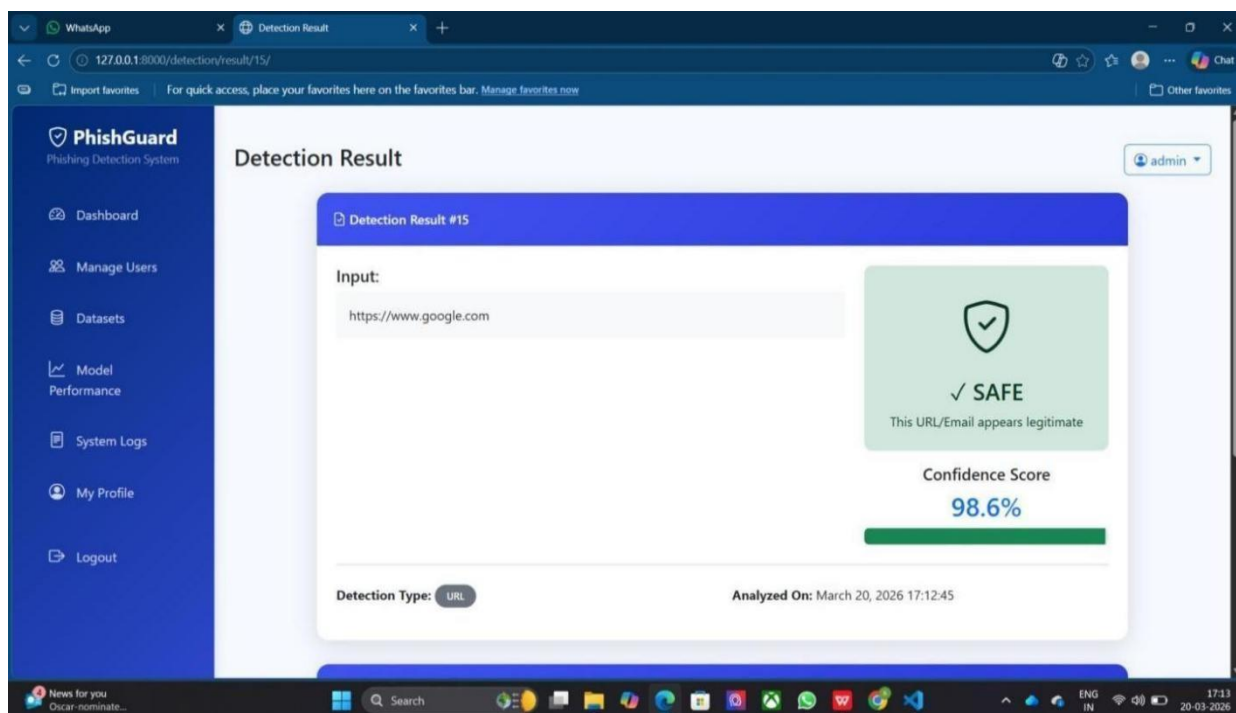
Hot days ahead
32°C

Search

ENG IN

17:13
20-03-2026





11. CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

The rapid growth of internet services, online banking, e-commerce, and digital communication has significantly increased the risk of phishing attacks. Traditional detection mechanisms such as blacklist-based filtering and static rule-based systems are no longer sufficient to handle modern phishing techniques. These conventional approaches fail to detect zero-day attacks and require constant manual updates, leading to reduced adaptability and high false-positive rates.

This project presented an **Advanced Machine Learning Approach for Phishing Website Identification using Meta-Ensemble Methods** to address these limitations. The proposed system integrates multiple intelligent classifiers—Artificial Neural Network (ANN), Bagging K- Nearest Neighbour (KNN), and Logistic Regression as a meta-classifier—within a stacking ensemble framework. ANN captures complex nonlinear global patterns in phishing datasets, Bagging KNN identifies local similarities and reduces variance, and Logistic Regression effectively combines predictions to generate a final optimized classification.

The system incorporates structured feature extraction from URLs, domain properties, hyperlink behavior, and webpage characteristics. Additionally, SMOTE is applied to handle class imbalance, ensuring fair and accurate model training. Cross-validation techniques further enhance model generalization and reliability.

Experimental evaluation demonstrates that the hybrid meta-ensemble model achieves higher accuracy, improved precision, better recall, and reduced false positives compared to individual classifiers. The real- time detection system implemented using Python and Django provides an interactive web interface where users can analyze suspicious URLs instantly. The inclusion of reporting and analytics modules further enhances system transparency and administrative monitoring.

Overall, the proposed framework successfully delivers a scalable, intelligent, and adaptive phishing detection system capable of identifying both known and unknown phishing attacks. The project contributes to strengthening cybersecurity infrastructure and protecting users from financial fraud, identity theft, and malicious cyber threats.

Future Enhancements

Although the proposed system demonstrates strong performance, several enhancements can further improve its efficiency, scalability, and adaptability.

One potential improvement is the integration of **deep learning-based URL embedding models**

such as recurrent neural networks (RNN) or transformer-based architectures to automatically learn semantic URL patterns without extensive manual feature engineering. This could improve detection accuracy against highly obfuscated phishing URLs.

Another enhancement involves deploying the system as a **browser extension or email filtering plugin**. This would allow real-time detection directly within web browsers or email clients, providing proactive protection before users access malicious websites. The system can also be extended with **real-time threat intelligence integration**, where it connects to external cybersecurity feeds and continuously updates phishing signatures and domain reputation data. This would improve adaptability against rapidly evolving threats.

Cloud-based deployment and containerization using technologies such as Docker and Kubernetes can enhance scalability, allowing the system to handle large-scale enterprise environments and high user traffic efficiently.

Further research can focus on **Explainable AI (XAI)** techniques to provide transparent and interpretable predictions. By explaining why a URL is classified as phishing, user trust and model accountability can be significantly improved.

Additionally, incorporating **continuous learning mechanisms** can allow the model to update itself automatically using new phishing data, ensuring long-term sustainability without manual retraining.

In conclusion, the proposed phishing detection system lays a strong foundation for intelligent cybersecurity solutions. With future enhancements in deep learning, explainability, cloud scalability, and real-time deployment, the framework can evolve into a comprehensive enterprise-level phishing prevention system capable of combating sophisticated and emerging cyber threats effectively.

6. REFERENCES

1. Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. (2009). **Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs.** *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'09)*, 1245–1254.
2. Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). **Predicting phishing websites based on self-structuring neural network.** *Neural Computing and Applications*, 25(2), 443–458.
3. Aburrous, M., Hossain, M. A., Dahal, K., & Thabtah, F. (2010). **Intelligent phishing detection system for e-banking using fuzzy data mining.** *Expert Systems with Applications*, 37(12), 7913–7921.
4. Wolpert, D. H. (1992). **Stacked Generalization.** *Neural Networks*, 5(2), 241–259.
5. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). **SMOTE: Synthetic Minority Over-sampling Technique.** *Journal of Artificial Intelligence Research*, 16, 321–357.
6. Huang, C., & Yao, Y. (2018). **URLNet: Learning a URL representation with deep learning for malicious URL detection.** *IEEE International Conference on Communications (ICC)*, 1–6.
7. Sahoo, D., Liu, C., & Hoi, S. C. H. (2017). **Malicious URL Detection using Machine Learning: A Survey.** *arXiv preprint arXiv:1701.07179*.
8. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). **Machine learning based phishing detection from URLs.** *Expert Systems with Applications*, 117, 345–357.
9. Jain, A. K., Duin, R. P. W., & Mao, J. (2000). **Statistical Pattern Recognition: A Review.** *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(1), 4–37.
10. Goodfellow, I., Bengio, Y., & Courville, A. (2016). **Deep Learning.** MIT Press, Cambridge,

7. BIBLIOGRAPHY

1. Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. (2009). *Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs*. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
2. Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). *Phishing Website Detection using Artificial Neural Networks*. International Journal of Computer Applications.
3. Aburrous, M., Hossain, M. A., Dahal, K., & Thabtah, F. (2010). *Intelligent Phishing Detection System for E-Banking using Fuzzy Data Mining*. Expert Systems with Applications.
4. Wolpert, D. H. (1992). *Stacked Generalization*. Neural Networks Journal.
5. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research.
6. Huang, C., & Yao, Y. (2018). *Deep Learning-Based Phishing Detection Using URL and Webpage Features*. IEEE Access.
7. Sahoo, D., Liu, C., & Hoi, S. C. H. (2017). *Malicious URL Detection using Machine Learning: A Survey*. arXiv Preprint.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
9. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques*. Morgan Kaufmann.
10. Documentation of Python Libraries:
 - TensorFlow Documentation: <https://www.tensorflow.org>
 - Scikit-learn Documentation: <https://scikit-learn.org>
 - Pandas Documentation: <https://pandas.pydata.org>